

PROJECT REPORT
on
**Addressing Conditional Shift in Graph
Data: An Empirical Study of Distance
Metrics**

Submitted By
Amitesh Puri
REG. NO: 233102023

In partial fulfillment of the requirement
for the award of degree
MASTER OF SCIENCE
in
APPLIED MATHEMATICS AND COMPUTING

Under the guidance of
Dr. Akansha
Assistant Professor



MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

Submitted To
Department of Mathematics
Manipal Institute of Technology

DECLARATION

I declare that the project entitled “**Addressing Distributional Shift in Graph Data: An Empirical Study of Distance Metrics**” for completion of my master degree has not been submitted in any other institute or university for the award of any other degree or diploma.

Amitesh Puri

REG.No.: 233102023

Department of Mathematics

Manipal Institute of Technology

MAHE

Place: Manipal

Date: May 10, 2025

CERTIFICATE



MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

This is to certify that the project entitled “Addressing Distributional Shift in Graph Data: An Empirical Study of Distance Metrics” is the work carried out by **Amitesh Puri** of M.Sc. Applied Mathematics and Computing, Manipal Institute of Technology, MAHE, Manipal, during the year 2023-2025, in partial fulfillment of the requirements of the award of the degree of **Master of Science in Applied Mathematics and Computing**.

Akansa
10/5/25

Dr. Akansha

Assistant Professor

Department of Mathematics

Manipal Institute of Technology

MAHE

Project Guide

Dr. Syam prasad Kuncham

Dr. Syam prasad Kuncham

Professor and Head

Department of Mathematics

Manipal Institute of Technology

MAHE

Place: Manipal

Date: May 10, 2025

ACKNOWLEDGEMENT

I would like to take this opportunity with great pleasure to acknowledge and appreciate all those who helped and guided me throughout the project.

I wish to thank my project guide **Dr. Akansha**, Associate Professor, Department of Mathematics, Manipal Institute of Technology, MAHE for helping me choose this very topic and constantly mentoring me in building up the project and giving necessary advice.

I would like to express my gratitude to **Dr. Syam prasad Kuncham**, Head of Department, Department of Mathematics, Manipal Institute of Technology, MAHE, for his support.

I also like to acknowledge the enthusiastic and valuable support of all the faculty members of the Mathematics Department, in completing my project.

Amitesh Puri

Contents

1	Introduction	5
2	Background	7
2.1	Node Classification	9
2.2	Message Passing	10
2.3	Distributional Shifts and Out-of-Distribution Generalization	12
2.4	Uncertainty Quantification (UQ)	13
2.5	Conformal Prediction (CP)	13
2.6	Distance and Divergence Metrics as Regularizers	14
1	Graph Neural Networks	16
1	Graph Convolutional Networks (GCN)	17
2	GraphSage	19
3	Graph Attention Networks (GAT)	21
4	Deep Adaptive Graph Neural Network (DAGNN)	24
5	APPNP	27
2	Distributional Shifts	31
1	Taxonomy of Distributional Shifts in Graph Learning	32
1.1	Covariate Shift :	32
1.2	Conditional Shift :	33
1.3	Structural Shift :	34

3	Metrics	36
1	Cross-Entropy(CE)	36
1.1	Properties of Cross-Entropy	37
2	Kullback-Leibler Divergence (KLD)	38
2.1	Properties of KLD	39
3	Jensen-Shannon Divergence (JSD)	40
3.1	Properties of JSD	40
4	Maximum Mean Discrepancy (MMD)	41
5	Central Moment Discrepancy (CMD)	42
4	Conformal Prediction	46
5	Methodology	49
0.1	Graph Neural Network Framework	50
0.2	Regularization	50
0.3	Our Proposed Regularization Framework	51
0.4	Training Procedure	52
0.5	Configuration	53
6	Experiments	55
0.1	Biased Training Set Creation	55
0.2	Datasets :	56
0.3	Models and Implementation :	57
7	Results	58
0.1	Accuracy Comparison	58
0.2	Coverage Results	59
0.3	Prediction Set Size	63

8	Ablation Study	65
1	Cora	65
1.1	Accuracy vs. Methods	65
1.2	Accuracy vs. Prediction Set Size (Fix Method, Vary Model)	67
1.3	Accuracy vs. Prediction Set Size (Fix Model, Vary Methods)	69
1.4	Method vs. Coverage (Fix Model)	71
1.5	Method vs. Prediction Set Size (Fix Model, Vary Methods)	73
1.6	Prediction Set Size vs. Methods	74
2	Citeseer	75
2.1	Accuracy vs. Method	75
2.2	Accuracy vs. Prediction Set Size (Fix Method, Vary Model)	77
2.3	Accuracy vs. Prediction Set Size (Fix Model and Vary Methods)	79
2.4	Method vs. Coverage (Fix Model)	80
2.5	Method vs. Prediction Set Size (Fix Model, Vary Methods)	82
2.6	Prediction Set Size vs. Methods (Bar Plot)	83
9	Conclusion	85
A	Appendix	86
1	Hyperparameters	86

Abstract

Graph Neural Networks (GNNs) have emerged as powerful tools for learning from graph-based data. Despite their potential, the practical deployment of current state-of-the-art (SOTA) GNNs in real-world scenarios faces significant hurdles. Two major interconnected limitations are particularly prominent: first, real-world data often exhibits distributional shifts between training and inference phases, and most GNNs are highly sensitive to these shifts, leading to a drastic degradation in their performance. Second, this sensitivity directly impacts the reliability of GNNs, which is a critical concern for their adoption in high-stakes applications such as the medical and financial domains. In this study, we address these limitations by investigating methods to improve GNN performance and quantify their uncertainty under such distributional shifts. We propose the integration of several distance or divergence metrics—namely Kullback-Leibler Divergence (KLD), Jensen-Shannon Divergence (JSD), Maximum Mean Discrepancy (MMD), and Central Moment Discrepancy (CMD)—as regularization terms during GNN training. The core hypothesis is that by encouraging the model to reduce these distributional discrepancies, the proposed methods will significantly improve GNN generalization in the presence of data shifts and, as a result, enhance their reliability. To validate our approach, this work conducts experiments on node classification benchmark datasets. These experiments aim to provide empirical evidence and insights into the efficacy of using distance metrics as regularizers. Furthermore, we utilize conformal prediction methods for robust uncertainty quantification of GNNs operating under these distributional shifts.

1 Introduction

Graph Neural Networks (GNNs) [1, 2, 3, 4, 5, 6] have achieved remarkable success in various graph-based tasks, particularly node classification, by effectively capturing relational and structural information from data assumed to be independent and identically distributed (IID). However, a significant and persistent challenge arises in real-world scenarios where GNNs inevitably encounter out-of-distribution (OOD) settings. In such cases, mismatches between training and inference data distributions—referred to as distributional shifts—can cause GNN performance to decline sharply, limiting their generalization capabilities [7]. This sensitivity is a major impediment to deploying GNNs reliably in high-stakes applications. For instance, in medicine, an inaccurate diagnostic prediction due to unforeseen data shifts could have severe patient outcomes, while in finance, flawed risk assessments could lead to significant monetary losses, underscoring the critical need for robust and trustworthy GNNs.

These distributional shifts [8, 9, 10], where the training data differs from the test data, generally fall into two main categories. The first, covariate shift [8], occurs when the characteristics of the input data change, but the underlying relationship between inputs and outputs remains the same. In contrast, conditional shift (or concept shift) [9] happens when the input data characteristics might stay consistent, but the fundamental rules or patterns connecting inputs to their corresponding outputs change. Conditional shifts are especially problematic because they directly challenge what the GNN has learned, leading to significant uncertainty about the reliability of its predictions [11, 12]. This situation forces us to ask the crucial question: ‘how good are your predictions?’ [13, 14], highlighting the need not only for more robust models but also for trustworthy ways to measure their uncertainty.

Recent work by Akansha et al. (2024) [15] made valuable progress by

demonstrating the effectiveness of Conformal Prediction (CP) [16, 17, 18, 19] as a framework for providing statistically valid uncertainty quantification and predictive intervals for GNN-based node classification, primarily assuming data exchangeability. While CP offers rigorous uncertainty bounds, its practical utility can be diminished if the underlying model’s predictions become grossly inaccurate due to severe distributional shifts, potentially leading to overly wide and less informative prediction sets. Building upon this foundation, our study investigates a complementary strategy aimed at first bolstering the GNN’s intrinsic robustness to these shifts, thereby providing a more reliable base for subsequent uncertainty quantification. We propose the integration of a combination of distinct distance and loss metrics—specifically, Cross-Entropy (CE) loss, Kullback-Leibler Divergence (KLD) [20], and Central Moment Discrepancy (CMD) [21]—as regularization terms during GNN training. Our rationale is that these metrics target unique aspects of distributional discrepancies: CE focuses on classification accuracy, KLD aims to align predicted and actual conditional label distributions (crucial for conditional shifts), and CMD captures structural differences in feature distributions. By synergistically combining these regularizers, we hypothesize an enhanced GNN generalization and robustness, particularly when faced with challenging conditional shifts. We then employ Conformal Prediction methods to quantify the uncertainty of these regularized GNNs, expecting more informative and reliable uncertainty estimates due to the improved underlying model.

Our main contributions are as follows:

1. We propose and evaluate a novel regularization framework for GNNs that integrates a combination of specific distance, divergence, and loss metrics (e.g., Cross-Entropy, KLD, and CMD) to mitigate the impact of distributional shifts, with a particular focus on conditional shifts.

2. Through extensive experimentation on node classification benchmarks, we demonstrate the efficacy of this combined regularization approach in significantly improving GNN accuracy and generalization under various challenging shift conditions.
3. We conduct a detailed ablation analysis to isolate the impact of conditional shifts and rigorously assess the benefits of our proposed method on model accuracy, calibration (via coverage), and the informativeness (size) of prediction sets generated by Conformal Prediction.

This report is structured as follows: Chapter 1 provides a detailed overview of Graph Neural Networks. Chapter 2 discusses the challenges of distributional shifts in machine learning, particularly for GNNs. Chapter 3 presents the metrics as regularizers. Chapter 4 introduces the principles of Conformal Prediction for uncertainty quantification. Chapter 5 presents our proposed methodology for improving GNN robustness and quantifying uncertainty using distance metric regularization and CP. Chapters 6 ,7 and 8 detail our experimental setup, results, and analysis, followed by conclusions in Chapter 9.

2 Background

This chapter lays the foundational groundwork necessary to understand the context and contributions of this report. We begin by introducing Graph Neural Networks (GNNs), their operational principles, and specifically detail the task of node classification. Subsequently, we delve into the challenge of distributional shifts in machine learning, particularly their impact on GNN performance. The critical need for Uncertainty Quantification (UQ) in such scenarios is then discussed, leading into an overview of Conformal Prediction (CP) as a robust framework for achieving this. Finally, we introduce the

concept of using distance and divergence metrics as regularization tools to enhance model robustness.

Graphs are ubiquitous data structures capable of representing complex relationships between entities. A graph G can be formally defined as $G = (V, E)$, where $V = \{v_1, v_2, \dots, v_n\}$ represents the set of nodes (or vertices), and $E \subseteq V \times V$ denotes the set of edges connecting these nodes. In practical applications, each node v_i is often associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^d$, while edges $(v_i, v_j) \in E$ may carry their own attribute information represented by $\mathbf{e}_{ij} \in \mathbb{R}^c$. For instance, in a social network, nodes represent users with features like age or interests, and edges represent friendships. GNNs have emerged as a powerful class of neural networks specifically designed to perform inference on data structured as graphs [22, 23].

The core operational principle behind most GNNs is **message passing** or **neighborhood aggregation** [24]. In this paradigm, each node iteratively updates its feature representation (or embedding) by aggregating information from its direct neighbors and combining it with its own current representation. This process typically involves multiple layers, allowing information to propagate across the graph. After several rounds of aggregation, the resulting node embeddings capture both the node’s own features and the structural information from its local graph neighborhood. These rich embeddings can then be used for various downstream tasks.

While numerous GNN architectures exist, prominent examples include:

- **Graph Convolutional Networks (GCNs)** [25], which generalize the convolution operation to graph data, often using a spectral or spatial approach for neighborhood aggregation.
- **Graph Attention Networks (GATs)** [26], which incorporate attention mechanisms to assign different levels of importance to different neighbors

during the aggregation process.

- **GraphSAGE** [27], which learns aggregator functions (e.g., mean, LSTM, pooling) to sample and aggregate features from a node’s local neighborhood, enabling inductive learning on unseen nodes.

2.1 Node Classification

Node classification is a fundamental task in graph machine learning and is the primary focus of this report. Given a graph where some nodes have known labels (e.g., categories, types), the goal of node classification is to predict the labels of the remaining unlabeled nodes or all nodes in the graph [28].

In a typical GNN-based node classification setup:

1. Each node $v_i \in V$ is initially associated with a feature vector $\mathbf{x}_i$.
2. The GNN processes the graph structure and node features through its layers, as described by the message passing paradigm, to compute a final embedding $\mathbf{h}_v$ for each node v . This embedding $\mathbf{h}_v$ is designed to capture relevant information from the node’s local neighborhood that is useful for predicting its label.
3. A classifier, often a simple feed-forward neural network (e.g., a linear layer followed by a softmax activation function), is applied to the final node embedding $\mathbf{h}_v$ to predict the probability distribution over possible labels for node v .
4. The GNN model is trained by minimizing a loss function, typically a cross-entropy loss, between the predicted label distributions and the true labels for the nodes in the training set.

Node classification has numerous applications, such as predicting the function of proteins in a protein-protein interaction network [29], identifying fraudulent users in a transaction graph [30], or classifying documents in a citation network [31]. The performance of GNNs on this task often relies heavily on the quality of learned node representations and their ability to generalize from labeled to unlabeled nodes.

Other common tasks addressed by GNNs include link prediction (predicting whether an edge exists between two nodes) and graph classification (predicting a label for an entire graph).

2.2 Message Passing

The fundamental principle behind Graph Neural Networks (GNNs) is message passing, where nodes iteratively exchange and aggregate information from their neighbors to capture graph structure and node features. This process is formalized as:

$$\mathbf{h}_v^{(t+1)} = \text{UPDATE}(\mathbf{h}_v^{(t)}, \text{AGGREGATE}(\{\mathbf{h}_u^{(t)} : u \in \mathcal{N}(v)\})), \quad (1)$$

where $\mathbf{h}_v^{(t)}$ represents the feature vector of node v at iteration t , $\mathcal{N}(v)$ is the neighborhood of v , AGGREGATE combines information from neighbors, and UPDATE refines the representation of v .

To ensure robustness and generalization, the AGGREGATE function must be *permutation-invariant*, meaning the output remains unchanged regardless of the order in which neighbors are processed. For example, summing or averaging neighbor features satisfies this property, while concatenation does not. Permutation invariance is crucial because the graph structure is inherently unordered, and a consistent node representation should not depend on an

arbitrary ordering of neighbors.

At each layer k , node representations $\mathbf{h}_v^{(k)}$ are updated using:

$$\mathbf{m}_v^{(k)} = \text{AGGREGATE}(\{\mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v)\}), \quad \mathbf{h}_v^{(k)} = \sigma(W^{(k)}\mathbf{m}_v^{(k)} + b^{(k)}), \quad (2)$$

where AGGREGATE (e.g., mean, sum, or max) aggregates information from neighbors, $W^{(k)}$ and $b^{(k)}$ are learnable parameters, and $\sigma(\cdot)$ is an activation function. This iterative update enables nodes to incorporate multi-hop dependencies and broader context within the graph.

The adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ provides a mathematical representation of the graph structure, where $\mathbf{A}_{ij} = 1$ if an edge exists between nodes v_i and v_j , and $\mathbf{A}_{ij} = 0$ otherwise. For weighted graphs, $\mathbf{A}_{ij}$ may contain real values indicating edge weights. The degree matrix $\mathbf{D}$ is typically defined as a diagonal matrix where $\mathbf{D}_{ii} = \sum_j \mathbf{A}_{ij}$, representing the number of connections for each node.

GNNs are broadly categorized into spectral and spatial methods. Spectral approaches, such as those based on the graph Laplacian $L = D - A$, define convolutional filters in the frequency domain. Spatial methods, on the other hand, directly operate on the neighborhood of each node. For instance, a Graph Convolutional Network (GCN) layer can be expressed as:

$$H^{(k)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(k-1)}W^{(k)}\right), \quad (3)$$

where $\tilde{A} = A + I$ is the adjacency matrix with self-loops and $\tilde{D}$ its degree matrix. By leveraging these principles, GNNs effectively capture both graph topology and node attributes, enabling their application to tasks like node classification, link prediction, and graph-level regression.

2.3 Distributional Shifts and Out-of-Distribution Generalization

A fundamental assumption in much of classical machine learning is that the training data and the data encountered during testing or deployment are **independent and identically distributed (IID)**. However, this assumption rarely holds perfectly in real-world applications. A **distributional shift** occurs when there is a mismatch between the probability distribution of the training data (P_{train}) and that of the test/inference data (P_{test}) [8]. Such shifts can severely degrade the performance and reliability of machine learning models, including GNNs, a phenomenon known as the Out-of-Distribution (OOD) generalization problem [32].

Distributional shifts can manifest in several forms [9]:

- **Covariate Shift:** This occurs when the distribution of input features X changes between training and testing (i.e., $P_{\text{train}}(X) \neq P_{\text{test}}(X)$), but the conditional probability of the label Y given the features X remains the same (i.e., $P_{\text{train}}(Y|X) = P_{\text{test}}(Y|X)$).
- **Conditional Shift (or Concept Shift):** This is often a more challenging scenario where the conditional distribution of labels given features changes (i.e., $P_{\text{train}}(Y|X) \neq P_{\text{test}}(Y|X)$), even if the input distribution $P(X)$ might remain relatively stable.

GNNs can be particularly vulnerable to distributional shifts due to their reliance on graph structure and local neighborhood features. Changes in graph topology or feature distributions that were not seen during training can lead the model to make erroneous predictions [33, 34]. This highlights the critical need for GNNs that are not only accurate on IID data but also robust and reliable when faced with OOD scenarios.

2.4 Uncertainty Quantification (UQ)

When a machine learning model makes a prediction, it is crucial, especially in high-stakes applications (e.g., medical diagnosis, financial forecasting), to understand the confidence or certainty associated with that prediction. **Uncertainty Quantification (UQ)** aims to provide a measure of this trust [14, 12]. Uncertainty can arise from various sources, broadly categorized as:

- **Aleatoric Uncertainty:** This is inherent randomness or noise in the data itself.
- **Epistemic Uncertainty:** This arises from the model’s lack of knowledge due to limited training data or an inappropriate model structure.

Standard outputs from many neural networks, such as the softmax scores in classification, are often miscalibrated and do not serve as reliable uncertainty measures, especially under distributional shifts where models might become “confidently wrong” [35]. Therefore, dedicated UQ methods are essential for building trustworthy AI systems.

2.5 Conformal Prediction (CP)

Conformal Prediction (CP) is a powerful, distribution-free (or assumption-light) framework for UQ that provides statistically rigorous guarantees on its prediction sets or intervals [16, 17, 18]. Instead of outputting a single point prediction, CP outputs a set of possible labels (for classification) or a range of values (for regression) that is guaranteed to contain the true outcome with a user-specified probability (e.g., 95%), under the assumption of data exchangeability.

The core mechanism of CP involves:

1. **Nonconformity Measure:** Defining a score that quantifies how “atypical” a data point is.

2. **Calibration:** Using a separate calibration dataset, nonconformity scores are computed.
3. **Prediction:** For a new test instance, a p-value is computed for each tentative prediction.
4. **Prediction Set:** All labels whose p-values exceed a predefined significance level α are included.

CP offers the significant advantage of providing marginal coverage guarantees without strong assumptions about the underlying data distribution or the model itself. This makes it particularly attractive for applications requiring reliable uncertainty estimates, such as those explored in this report [36].

2.6 Distance and Divergence Metrics as Regularizers

Regularization in machine learning refers to techniques used to prevent overfitting and improve the generalization ability of a model by adding a penalty term to the loss function. In the context of improving robustness to distributional shifts, specific distance or divergence metrics can be employed as regularizers to encourage the model to learn representations or make predictions that are invariant or less sensitive to these shifts.

Several metrics can quantify the discrepancy between probability distributions or feature representations:

- **Kullback-Leibler Divergence (KLD):** Measures the difference between two probability distributions [20].
- **Jensen-Shannon Divergence (JSD):** A symmetric and smoothed version of KLD [37].

- **Maximum Mean Discrepancy (MMD):** A non-parametric metric comparing mean embeddings in an RKHS [38].
- **Central Moment Discrepancy (CMD):** Compares distributions by matching their central moments [39].

By incorporating these metrics into the GNN’s training objective, the aim is to explicitly penalize deviations between, for example, the distributions of learned node representations from the training data and potentially shifted test data. This can guide the GNN towards learning more robust features that generalize better under shifts, thereby providing a more stable foundation for subsequent uncertainty quantification using methods like Conformal Prediction. The synergy between such regularization and CP forms a core aspect of the methodology proposed in this report.

Chapter 1

Graph Neural Networks

In this chapter we investigate the theoretical foundations and computational architectures of Graph Neural Networks (GNNs), which represent an important advancement in moving beyond traditional neural networks designed for grid-like data. GNNs provide a principled framework for learning representations that preserve and exploit the relational inductive biases inherent in graph-structured information. The following sections explore the evolution of graph-based learning algorithms, examining their mathematical formulations and computational properties. We analyze how these architectures integrate node features with topological information through specialized message-passing operations. This analysis provides essential context for understanding these models and their respective contributions to the field. Through systematic exploration of these models, we establish the necessary theoretical groundwork for the research contributions presented in later chapters, particularly regarding the incorporation of specialized regularization techniques within graph learning frameworks.

1 Graph Convolutional Networks (GCN)

Graph Convolutional Networks (GCNs), introduced by Kipf et al. [40], are a foundational approach for leveraging graph-structured data in neural networks. GCNs operate by propagating node features through the graph structure, enabling representation learning that considers both feature and topological information. The key operation in GCNs is the layer-wise propagation rule, which updates node embeddings by aggregating information from local neighborhoods. The rule is defined as:

$$H^{(l+1)} = \sigma(\tilde{A}H^{(l)}W^{(l)}),$$

where $H^{(l)} \in \mathbb{R}^{n \times d_l}$ is the feature matrix at layer l , $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ is a learnable weight matrix, and $\sigma(\cdot)$ is a non-linear activation function such as ReLU, $\text{ReLU}(x) = \max(0, x)$. Here, $\tilde{A} = D'^{-\frac{1}{2}}A'D'^{-\frac{1}{2}}$ is the normalized adjacency matrix, where $A' = A + I_n$ is the adjacency matrix with added self-loops, and D' is the corresponding degree matrix with $D'_{ii} = \sum_j A'_{ij}$. The input to the network is $H^{(0)} = X$, where $X \in \mathbb{R}^{n \times d_0}$ represents the input node features.

GCNs are theoretically motivated by spectral graph convolutions, which involve transforming graph signals into the Fourier domain. The graph Laplacian L is defined as $L = I_n - D^{-\frac{1}{2}}AD^{-\frac{1}{2}}$ and has an eigendecomposition $L = U\Lambda U^\top$, where U contains eigenvectors and Λ is a diagonal matrix of eigenvalues. A spectral convolution on a graph is given by:

$$g_\theta * x = Ug_\theta(\Lambda)U^\top x,$$

where $g_\theta(\Lambda)$ is a spectral filter parameterized by θ . Computing this directly is computationally prohibitive for large graphs due to the cost of eigenvalue

decomposition.

To circumvent this, [41] proposed approximating $g_\theta(\Lambda)$ using Chebyshev polynomials, which avoids the eigenvalue decomposition and reduces complexity to $\mathcal{O}(|E|)$. By limiting the order of the Chebyshev polynomial to $K = 1$, [40] further simplified the spectral convolution to a first-order approximation:

$$H^{(l+1)} = \sigma(\tilde{A}H^{(l)}W^{(l)}).$$

This simplification retains efficiency and scalability while maintaining the model’s ability to learn expressive node embeddings. The resulting GCN is equivalent to applying a localized, linear operator to each node’s neighborhood, followed by a non-linear transformation.

A typical GCN for semi-supervised classification consists of two layers. The output is computed as:

$$Z = \text{softmax}(\tilde{A} \text{ReLU}(\tilde{A}XW^{(0)})W^{(1)}),$$

where $Z \in \mathbb{R}^{n \times C}$ gives the class probabilities for C classes. The model is trained by minimizing the cross-entropy loss:

$$\mathcal{L} = - \sum_{i \in \mathcal{Y}_L} \sum_{c=1}^C Y_{ic} \log Z_{ic},$$

where $\mathcal{Y}_L$ is the set of labeled nodes, and Y is the one-hot encoded label matrix.

The computational complexity per layer of a GCN is $\mathcal{O}(|E|d_{l+1})$, where $|E|$ is the number of edges, making it scalable for large sparse graphs. The combination of efficiency, theoretical grounding, and performance on real-world tasks has established GCNs as a standard tool in graph-based machine learning.

2 GraphSage

GraphSAGE (Graph Sample and Aggregate) is an inductive framework designed to learn node embeddings by aggregating information from local neighborhoods, enabling generalization to unseen nodes and graphs [2]. Unlike transductive methods such as the Graph Convolutional Network (GCN) [40], which require the entire graph to be available during training, GraphSAGE is capable of working in dynamic or evolving graph settings by leveraging node features for embedding generation. The framework achieves this by iteratively sampling and aggregating information from a node’s neighborhood to produce embeddings that capture both the structural and feature-based properties of the graph.

In GraphSAGE, the embedding of a node at layer k , denoted $h_v^{(k)}$, is computed through a three-step process: neighbor sampling, feature aggregation, and transformation. First, a fixed-size subset of the node’s neighbors $\mathcal{N}(v)$ is sampled uniformly to limit computational complexity. Next, the feature representations of these neighbors, $h_u^{(k-1)}$, from the previous layer are aggregated using a differentiable, symmetric function denoted as AGGREGATE. This ensures permutation invariance in the unordered neighborhood structure. Finally, the aggregated neighborhood representation is concatenated with the node’s current representation, $h_v^{(k-1)}$, and transformed via a learnable weight matrix $W^{(k)}$, followed by a non-linear activation function σ . The process is formally expressed as:

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \text{CONCAT} \left(h_v^{(k-1)}, \text{AGGREGATE} \left(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\} \right) \right) \right),$$

where $h_v^{(0)}$ is initialized to the input node features.

GraphSAGE employs multiple types of AGGREGATE functions to suit different application scenarios: - Mean Aggregator: Computes the element-wise

mean of the neighbor representations, analogous to the convolutional operator in GCNs, but adapted for inductive tasks.

- LSTM Aggregator: Applies a Long Short-Term Memory (LSTM) network to sequentially process the neighbor representations. Since graph neighbors lack a natural ordering, the sequence is generated randomly, introducing variability.
- Pooling Aggregator: Transforms each neighbor’s features using a feedforward neural network, followed by an element-wise max-pooling operation. This approach captures salient features across the neighborhood.

The iterative process aggregates information from increasingly larger multi-hop neighborhoods as the number of layers K increases, allowing nodes to gain a broader contextual understanding. To ensure scalability, GraphSAGE uses fixed-size neighbor sampling, reducing the memory and computational footprint to $O(K \cdot S)$, where S is the sample size per layer. This design allows the framework to handle large-scale graphs efficiently.

GraphSAGE’s parameters, including the weight matrices $W^{(k)}$ and the aggregation function, are learned via stochastic gradient descent using a graph-based loss function. For unsupervised learning, the objective encourages nodes close in the graph structure (e.g., co-occurring in random walks) to have similar embeddings while distinguishing distant nodes:

$$\mathcal{L} = -\log \sigma(z_u^\top z_v) - \mathbb{E}_{v_n \sim P_n(v)} [\log \sigma(-z_u^\top z_{v_n})],$$

where σ is the sigmoid function, P_n is a negative sampling distribution, and z_u is the final embedding of node u after K layers.

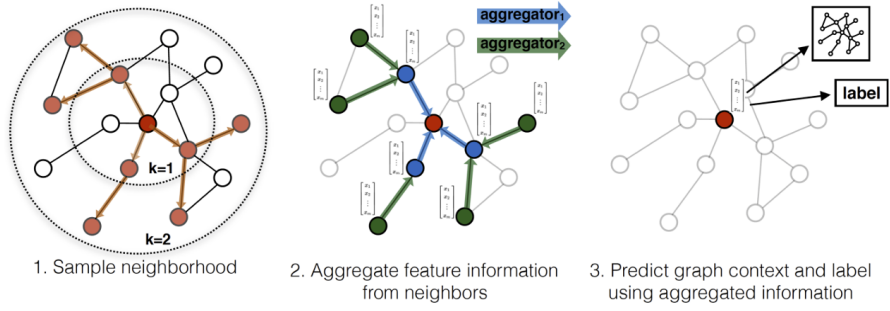


Figure 1: Visual illustration of the GraphSAGE sample and aggregate approach.

Figure 1.1: The GraphSAGE framework: Nodes iteratively aggregate and transform features from their local neighborhoods using symmetric functions such as mean, LSTM, or pooling. The embeddings are updated at each layer, capturing both local and global graph information.

Source: Adapted from [2].

By integrating neighbor sampling, aggregation, and transformation, GraphSAGE provides a scalable and flexible method for learning node representations. Its conceptual ties to the Weisfeiler-Lehman graph isomorphism test offer theoretical insights into its ability to capture the topological and feature-based intricacies of graphs, while its practical design supports diverse applications in social networks, biological graphs, and knowledge graphs.

3 Graph Attention Networks (GAT)

Graph Attention Networks (GAT), introduced by P. Velkovic et al.[4], represent a significant advancement in graph-based learning by integrating the self-attention mechanism to improve the aggregation of node features in a graph. Unlike previous methods that relied on fixed aggregation schemes, this architecture adaptively learn the importance of neighboring nodes for feature propagation, offering a more flexible and expressive approach to graph representation learning.

GAT utilizes the graph attentiona layer(s), which transforms input node features into higher-level representations while attending to relevant neighbors.

Let the input to the layer be a set of node features $\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_N]$, where $\mathbf{h}_i \in \mathbb{R}^F$ is the feature vector of node i , N is the number of nodes, and F is the feature dimension. The output of the layer is a new set of node features $\mathbf{H}' = [\mathbf{h}'_1, \mathbf{h}'_2, \dots, \mathbf{h}'_N]$, where $\mathbf{h}'_i \in \mathbb{R}^{F'}$.

The process begins with a shared linear transformation applied to all nodes:

$$\mathbf{h}'_i = \mathbf{W}\mathbf{h}_i, \quad \mathbf{W} \in \mathbb{R}^{F' \times F},$$

where $\mathbf{W}$ is a learnable weight matrix. Next, a self-attention mechanism computes attention coefficients for node pairs. The unnormalized attention coefficient between nodes i and j is given by:

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_j]),$$

where $\mathbf{a} \in \mathbb{R}^{2F'}$ is a learnable weight vector, $\|$ denotes concatenation, and LeakyReLU applies the activation function $\text{LeakyReLU}(x) = \max(0, x) + \alpha \min(0, x)$ with negative slope $\alpha = 0.2$.

To ensure comparability across nodes, the attention coefficients are normalized using the softmax function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})},$$

where $\mathcal{N}(i)$ denotes the neighborhood of node i (including i itself). The normalized coefficients α_{ij} quantify the relative importance of node j 's features for updating node i .

The final output feature for node i is computed as a weighted sum of the

transformed features of its neighbors:

$$\mathbf{h}'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W} \mathbf{h}_j \right),$$

where $\sigma(\cdot)$ is a non-linear activation function.

To improve stability and model expressiveness, GATs employ multi-head attention. Multiple independent attention mechanisms are executed in parallel, producing K sets of attention coefficients $\alpha_{ij}^{(k)}$ and weight matrices $\mathbf{W}^{(k)}$. The outputs of these heads are concatenated:

$$\mathbf{h}'_i = \parallel_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \mathbf{W}^{(k)} \mathbf{h}_j \right),$$

resulting in a feature vector of dimension $K \cdot F'$. In the final prediction layer, multi-head attention outputs are averaged instead of concatenated:

$$\mathbf{h}'_i = \frac{1}{K} \sum_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \mathbf{W}^{(k)} \mathbf{h}_j \right).$$

This approach allows GATs to model the importance of relationships between nodes dynamically, leading to improved performance on tasks such as node classification and graph representation learning.

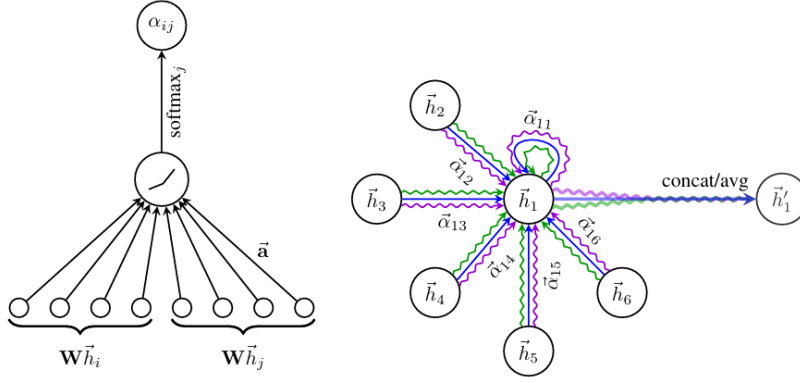


Figure 1.2: (Left) The attention mechanism $a(\mathbf{W}\mathbf{h}_i, \mathbf{W}\mathbf{h}_j)$ employed in the GAT model, parameterized by a weight vector $\mathbf{a} \in \mathbb{R}^{2F'}$, applying a LeakyReLU activation. (Right) An illustration of multi-head attention with $K = 3$ heads. Different arrow styles and colors represent independent attention computations. The aggregated features from each head are concatenated or averaged to compute the final representation of node h_1 .
Figure source: Adapted from [4].

4 Deep Adaptive Graph Neural Network (DAGNN)

Deep Adaptive Graph Neural Network (DAGNN) is a novel framework designed to overcome limitations in existing GNN architectures by decoupling representation transformation from propagation and incorporating an adaptive adjustment mechanism [42]. This architecture enables the application of large receptive fields without performance degradation and allows for node-specific balancing of local and global neighborhood information, yielding more discriminative node representations.

The mathematical formulation of DAGNN is expressed as follows:

$$Z = \text{MLP}(X), \quad H_\ell = A^\ell Z, \quad \ell = 1, 2, \dots, k,$$

$$H = \text{stack}(Z, H_1, \dots, H_k), \quad S = \sigma(Hs), \quad S = \text{reshape}(S),$$

$$X_{\text{out}} = \text{softmax}(\text{squeeze}(SH)),$$

where $X \in \mathbb{R}^{n \times d}$ is the input feature matrix, $Z \in \mathbb{R}^{n \times c}$ is the transformed feature matrix, and $H_\ell \in \mathbb{R}^{n \times c}$ represents the propagated feature representations at ℓ -hop depth. The propagation mechanism uses a symmetrically normalized adjacency matrix $A = D^{-\frac{1}{2}}(A + I)D^{-\frac{1}{2}}$, where I is the identity matrix and D is the degree matrix. $s \in \mathbb{R}^{c \times 1}$ is a trainable projection vector, and σ represents the sigmoid activation function. Operations such as stack, reshape, and squeeze are used to ensure compatibility of tensor dimensions during computation.

DAGNN consists of three main steps: transformation, propagation, and adaptive adjustment. In the transformation step, a shared multi-layer perceptron (MLP) is applied to the input feature matrix X , resulting in Z , which contains node-level information independent of structural context. This step leverages the universal approximation property of MLPs to transform features into a more expressive space. However, Z lacks structural information, which is incorporated in the next step.

In the propagation step, $H_\ell = A^\ell Z$ aggregates neighborhood information up to ℓ -hops. This captures structural information from subtrees of height ℓ rooted at each node. Larger values of ℓ allow the inclusion of more global context, but an excessively large ℓ risks diluting crucial local information. Hence, determining an appropriate ℓ for each node is critical.

To address this, DAGNN introduces an adaptive adjustment mechanism. A shared trainable vector s generates retainment scores through the operation $S = \sigma(Hs)$, determining the importance of features derived at different propagation layers. These scores balance the contribution of local and global neighborhood information for each node, enabling node-specific adjustments. By stacking and reshaping the propagated features, DAGNN ensures that the contributions from multiple layers are adaptively combined.

The framework is highly parameter-efficient since the propagation process itself does not introduce trainable parameters. Additionally, DAGNN’s end-to-end training integrates transformation, propagation, and adaptive adjustment into a unified optimization objective. The model is computationally efficient due to its sequential computation of $A^\ell Z$, reducing the time complexity to $O(n^2c)$ compared to $O(n^3)$ for traditional propagation approaches.

DAGNN is also memory-efficient, as it transforms features into a low-dimensional space early in the process. Unlike traditional architectures, it does not rely on fully connected layers as a classifier; instead, the final representations X_{out} are used directly for node classification. The loss function for labeled nodes is computed using the cross-entropy loss:

$$\mathcal{L} = - \sum_{i \in \mathcal{V}_L} \sum_{p=1}^c Y[i, p] \ln X_{\text{out}}[i, p],$$

where $\mathcal{V}_L$ is the set of labeled nodes, Y is the label indicator matrix, and c is the number of classes.

DAGNN’s innovative design, including the decoupling of representation transformation and propagation and the adaptive adjustment mechanism, ensures its capability to generate high-quality node embeddings from large and adaptive receptive fields. These features make it a robust choice for tasks requiring efficient and scalable node classification and graph representation learning.

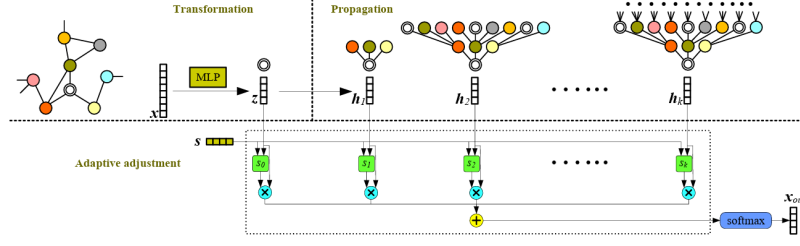


Figure 1.3: An illustration of the proposed Deep Adaptive Graph Neural Network (DAGNN) . For clarity, we show the pipeline to generate the prediction for one node. Notation letters are consistent with the mathematical formulation of DAGNN but bold lowercase versions are applied to denote representation vectors. s is the projection vector that computes retainment scores for representations generating from various receptive fields. s_0, s_1, s_2 , and s_k represent the retainment scores of z , h_1, h_2 , and h_k , respectively.

Figure source: Adapted from [42].

5 APPNP

Personalized Propagation of Neural Predictions (PPNP) builds upon the relationship between graph neural networks (GNNs) and the Personalized PageRank algorithm [43], which extends the classical PageRank by incorporating a teleport mechanism for node-specific focus [44]. In standard PageRank, the iterative propagation of influence across nodes is determined by the adjacency matrix, while the inclusion of a teleport vector in Personalized PageRank allows for localized focus, preserving the individuality of each node during propagation. This principle underpins PPNP, enabling the integration of node-level predictions with graph topology for semi-supervised classification.

PPNP generates predictions using a neural network, which operates independently on node features, followed by a propagation step based on the fully personalized PageRank mechanism. This separation of feature transformation from propagation allows the neural network to remain shallow while still benefiting from an effectively infinite receptive field during propagation. Mathematically, the

PPNP model is expressed as:

$$Z_{\text{PPNP}} = \text{softmax} \left((\mathbf{I}_n - (1 - \alpha)\mathbf{A})^{-1} H \right),$$

where $H = f(X)$ represents the node-wise predictions derived from a neural network f , $X \in \mathbb{R}^{n \times d}$ is the node feature matrix, $\mathbf{I}_n$ is the identity matrix, $\mathbf{A}$ is the adjacency matrix normalized with self-loops, and α (the teleport probability) controls the balance between local and global influence. The inverse term $(\mathbf{I}_n - (1 - \alpha)\mathbf{A})^{-1}$ captures the iterative aggregation of information across the graph, effectively encoding infinite-hop propagation. This mechanism enhances the model’s flexibility, allowing it to capture long-range dependencies without increasing the depth of the neural network, making it particularly advantageous in graph-based tasks where global context is critical.

However, direct computation of the inverse matrix $(\mathbf{I}_n - (1 - \alpha)\mathbf{A})^{-1}$ is computationally infeasible due to its dense nature, which scales quadratically with the number of nodes ($O(n^2)$). To address this, Approximate Personalized Propagation of Neural Predictions (APPNP) leverages a power iteration approach to approximate PPNP efficiently. APPNP avoids explicit matrix inversion, ensuring scalability while retaining the sparsity of the graph. The iterative formula is defined as:

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha)\mathbf{A}Z^{(k)} + \alpha H, \quad k = 0, 1, \dots, K - 1,$$

$$Z^{(K)} = \text{softmax} \left(Z^{(K-1)} \right),$$

where K denotes the number of power iterations and $Z^{(k)}$ represents the intermediate predictions after k iterations. In this formulation, the parameter α plays a crucial role in determining the trade-off between local and global

aggregation. Larger α values emphasize local node features, while smaller values encourage broader propagation. This iterative scheme, by avoiding the construction of a dense personalized PageRank matrix, reduces computational complexity to $O(K \cdot |E|)$, where $|E|$ is the number of edges in the graph. Additionally, the algorithm preserves memory efficiency by propagating sparse matrices.

The power iteration approach in APPNP connects the propagation step to a random walk with restarts, where each step probabilistically blends information from neighbors (via the adjacency matrix) with the node’s initial prediction (via the teleport vector). This ensures that node-specific predictions are retained throughout the propagation process. Unlike traditional message-passing frameworks, APPNP’s propagation does not introduce additional trainable parameters, making it parameter-efficient and well-suited for tasks requiring far-reaching neighborhood aggregation. Furthermore, the ability to adjust α provides flexibility to accommodate diverse graph structures, enabling effective performance across different domains such as citation networks, social graphs, and biological networks.

Another advantage of APPNP is its end-to-end trainability, where gradients flow seamlessly through the propagation scheme during backpropagation. This ensures that both the neural network predictions and the propagation process are optimized jointly, yielding better overall accuracy. Moreover, the model supports the substitution of $\mathbf{A}$ with other propagation matrices, such as the random-walk normalized adjacency matrix, allowing for further customization. By reformulating PPNP as a fixed-point iteration, APPNP bridges the theoretical principles of PageRank with the practical requirements of GNNs, offering a robust and efficient framework for graph learning tasks.

APPNP’s iterative refinement and efficient handling of graph sparsity make

it particularly powerful in scenarios requiring large-scale propagation. Unlike traditional GNNs that suffer from oversmoothing or require extensive parameters for deep architectures, APPNP achieves superior performance by separating the depth of feature transformation from the propagation depth. Its parameter efficiency, coupled with flexibility and scalability, positions APPNP as a significant advancement in graph-based semi-supervised learning.

Chapter 2

Distributional Shifts

Distributional shift refers to changes in the distribution of data between the training and testing (or inference) phases, expressed as:

$$P_{\text{train}}(X) \neq P_{\text{test}}(X).$$

In Graph Neural Networks (GNNs), this shift disrupts the learned mapping between graph structure, node features, and labels, leading to degraded model performance when deployed on unseen data. Let the final node representations:

$$N = \{n_1, n_2, \dots, n_n\}$$

represent the embeddings produced by the GNN for n nodes in a graph G . If the subset of labeled node representations $N_l \subset N$ used for training follows a different distribution than the unlabeled nodes encountered during inference, the model's performance may be affected due to this divergence.

The fundamental challenge of distributional shift stems from the core assumption in statistical learning theory that training and test data are drawn from identical distributions [45]. When this assumption is violated, models trained

to minimize empirical risk on the training distribution may fail to generalize effectively to the test distribution. This issue is particularly pronounced in graph-structured data, where complex interdependencies between nodes make the effects of distributional shifts more severe and challenging to mitigate.

1 Taxonomy of Distributional Shifts in Graph Learning

While distributional shift encompasses a broad spectrum of phenomena, this chapter focuses on three primary categories that are particularly relevant to graph neural networks: covariate shift, conditional shift (also known as concept shift), and structural shift. Each type presents unique challenges and requires specific strategies for detection and mitigation.

1.1 Covariate Shift :

Covariate shift occurs when the distribution of the input features changes between training and testing while the conditional relationship between features and labels remains unchanged, i.e.,

$$P_{\text{train}}(X) \neq P_{\text{test}}(X) \quad \text{but} \quad P_{\text{train}}(Y|X) = P_{\text{test}}(Y|X).$$

In GNNs, such shifts may arise from variations in node attributes across different graphs or domains.

Formally, if we denote the feature space as $\mathcal{X}$ and the label space as $\mathcal{Y}$, covariate shift implies that the marginal distributions $P_{\text{train}}(X)$ and $P_{\text{test}}(X)$ differ while the conditional distributions $P(Y|X)$ remain consistent across training and testing environments[46]. In graph-structured data, covariate

shift manifests in several ways. For example, the distribution of node features may change across different graphs or domains. In citation networks, the feature distribution of papers changes over time as research trends evolve. The connectivity patterns of nodes might vary between training and testing graphs, resulting in different degree distributions that affect message passing operations in GNNs[27]. Additionally, sensor noise or data collection inconsistencies can introduce systematic differences in feature quality between training and production environments[47].

Covariate shift in GNNs presents unique challenges because the interdependence between nodes means that shifts in one part of the graph can propagate through the network during message passing. Zhu et al.[48] demonstrated that even localized covariate shifts can affect global graph properties, causing cascading effects on GNN performance.

1.2 Conditional Shift :

Conditional shift occurs when the conditional distribution of labels given features differs between training and testing, even if the feature distributions remain consistent. Formally,

$$P_{\text{train}}(Y|X) \neq P_{\text{test}}(Y|X) \quad \text{while} \quad P_{\text{train}}(X) = P_{\text{test}}(X).$$

In GNNs, this can be observed when the label-generating process or graph semantics change between datasets.

This type of shift represents a fundamental change in the underlying relationship between node features and their corresponding labels. For example, the semantic meaning of connections between nodes may change over time. In financial transaction networks, patterns indicating fraudulent behavior evolve as perpetrators

adapt to detection systems. The interpretation of node features may depend on the broader context or domain, such that the same feature values may correspond to different semantic meanings when a GNN trained in one domain is applied to another[49]. Furthermore, the definition or distribution of labels may change over time, even if the feature distribution remains stable. In citation networks, the criteria for categorizing papers into research areas evolve as fields emerge and merge[4].

Conditional shift is particularly challenging because it cannot be addressed through traditional domain adaptation techniques that focus on aligning feature distributions [50]. Instead, it requires adaptation of the model’s decision boundaries or explicit modeling of the changing conditional distributions.

1.3 Structural Shift :

Although not included in the traditional taxonomy of distributional shifts, structural shift is particularly relevant to graph neural networks. Structural shift refers to changes in the underlying graph structure or topology between training and testing environments, while node features and labels may follow similar distributions. Formally, if we represent the graph structure through an adjacency matrix A , structural shift occurs when

$$P_{\text{train}}(A) \neq P_{\text{test}}(A).$$

[51]

Structural shifts can manifest in various ways. For instance, the overall connectivity of the graph may differ between training and testing, affecting the efficiency of message passing in GNNs[6]. The organization of nodes into communities or clusters may differ, changing the contextual information

available during aggregation steps[52]. Additionally, the tendency of similar nodes to connect (homophily) may vary across graphs, affecting the assumptions underlying many GNN architectures[53].

Structural shifts are particularly problematic for GNN architectures that rely heavily on neighborhood aggregation, as these operations implicitly encode assumptions about the graph structure[54].

Detecting and quantifying these distributional shifts is essential for understanding model performance degradation and developing appropriate mitigation strategies. In this work we focused on Conditiona Shift and several metrics and methodologies have been proposed for addressing this issue in graph data.

Chapter 3

Metrics

This chapter examines the theoretical foundations of metrics as they relate to our proposition to be used for regularization techniques in loss functions. By establishing the mathematical properties and statistical behaviour of these metrics, we provide the necessary context for understanding how regularizers impacts optimization landscapes and influence model characteristics. This foundational discussion serves as the analytical framework through which subsequent regularization implementations and their effects on model performance can be rigorously assessed.

1 Cross-Entropy(CE)

Cross-entropy is a measure from information theory that quantifies the average number of bits required to encode an event drawn from a probability distribution P when using a coding scheme optimized for a different probability distribution Q . In machine learning, it is widely used as a loss function, particularly in classification tasks, to measure the difference between the predicted probability distribution Q and the true distribution P .

The cross-entropy $H(P, Q)$ between a distribution P and a distribution Q

is defined as:

For discrete probability distributions P and Q with probability mass functions $p(x)$ and $q(x)$ respectively, defined on the same probability space $\mathcal{X}$:

$$H(P, Q) = - \sum_{x \in \mathcal{X}} p(x) \log q(x)$$

For continuous probability distributions P and Q with probability density functions $p(x)$ and $q(x)$ respectively:

$$H(P, Q) = - \int_{-\infty}^{\infty} p(x) \log q(x) dx$$

The logarithm base determines the units (nats for base e , bits for base 2).

Cross-entropy is always non-negative.

1.1 Properties of Cross-Entropy

- **Relationship to KLD and Entropy:** Cross-entropy is related to the Kullback-Leibler Divergence and the entropy of P , $H(P)$, by the equation: $H(P, Q) = D_{KL}(P||Q) + H(P)$.
- **Asymmetry:** Cross-entropy is generally asymmetric, meaning $H(P, Q) \neq H(Q, P)$.
- **Boundedness:** Bounded below by the entropy of P , $H(P)$. It is not bounded above in general, especially if $q(x)$ can be close to zero for values of x where $p(x) > 0$.
- **Differentiability:** The cross-entropy function is differentiable with respect to the parameters of the distribution Q (e.g., the output of a model), which is crucial for gradient-based optimization methods used in training neural networks.

- **Time Complexity:** Computing cross-entropy for discrete distributions over a space of size $|\mathcal{X}|$ takes $O(|\mathcal{X}|)$ time. For continuous distributions, it depends on the integration method. In practice, for empirical distributions (samples), it's often computed on mini-batches, taking time proportional to the batch size and the dimensionality of the data.

2 Kullback-Leibler Divergence (KLD)

The Kullback-Leibler Divergence (KLD), or relative entropy, is a measure of how one probability distribution P diverges from a second, reference probability distribution Q . It quantifies the information lost when Q is used to approximate P .

The KLD of P from Q , denoted $D_{KL}(P||Q)$, is defined as:

For discrete probability distributions P and Q with probability mass functions $p(x)$ and $q(x)$ respectively, defined on the same probability space $\mathcal{X}$:

$$D_{KL}(P||Q) = \sum_{x \in \mathcal{X}} p(x) \log \left(\frac{p(x)}{q(x)} \right)$$

For continuous probability distributions P and Q with probability density functions $p(x)$ and $q(x)$ respectively:

$$D_{KL}(P||Q) = \int_{-\infty}^{\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

The logarithm base is typically e or 2. The KLD is defined only if $q(x) > 0$ whenever $p(x) > 0$ (absolute continuity). A standard reference for KLD is "Elements of Information Theory" by Cover and Thomas [55].

2.1 Properties of KLD

- **Non-negativity:** $D_{KL}(P||Q) \geq 0$, with equality if and only if $P = Q$.

This is a consequence of Jensen's inequality.

- **Not a True Metric:** KLD is not a true metric because it does not satisfy two key properties of a metric:

- **Asymmetry:** In general, $D_{KL}(P||Q) \neq D_{KL}(Q||P)$. The order of P and Q matters.
- **Triangle Inequality:** The triangle inequality ($D(P, R) \leq D(P, Q) + D(Q, R)$) does not hold for KLD.

Despite not being a true metric, KLD is widely used because it has a strong theoretical foundation in information theory [55] and probability. It provides a meaningful measure of information gain or divergence between distributions. In optimization, minimizing $D_{KL}(P||Q)$ is often a well-posed problem that encourages Q to place high probability mass where P does.

- **Boundedness:** KLD can be infinite if there are points where $p(x) > 0$ and $q(x) = 0$. It is not bounded above in general.
- **Differentiability:** KLD is differentiable with respect to the parameters of Q (and sometimes P), making it suitable for gradient-based optimization.
- **Time Complexity:** Similar to cross-entropy, computing empirical KLD for discrete distributions or sampled continuous distributions takes time proportional to the size of the support or the number of samples and the dimensionality.

3 Jensen-Shannon Divergence (JSD)

The Jensen-Shannon Divergence (JSD) is a symmetric and smoothed version of the Kullback-Leibler Divergence. It is based on Jensen's inequality and the properties of Shannon entropy.

The JSD between two probability distributions P and Q is defined as:

$$D_{JS}(P||Q) = \frac{1}{2}D_{KL}\left(P||\frac{P+Q}{2}\right) + \frac{1}{2}D_{KL}\left(Q||\frac{P+Q}{2}\right)$$

where $M = \frac{P+Q}{2}$ is the mixture distribution.

3.1 Properties of JSD

- **Symmetry:** $D_{JS}(P||Q) = D_{JS}(Q||P)$. This is a direct consequence of its definition.
- **Boundedness:** JSD is bounded. For base-2 logarithms, $0 \leq D_{JS}(P||Q) \leq 1$. For base- e logarithms, $0 \leq D_{JS}(P||Q) \leq \log(2)$.
- **True Metric (for the square root):** While JSD itself does not satisfy the triangle inequality, its square root, $\sqrt{D_{JS}(P||Q)}$, is a true metric. This makes JSD (or its square root) suitable for applications where a metric is required, such as in clustering or defining distances in feature spaces.
- **Defined for Disjoint Supports:** Unlike KLD, JSD is well-defined even if P and Q have disjoint supports.
- **Differentiability:** JSD is differentiable, which allows its use in gradient-based optimization.

- **Time Complexity:** Similar to KLD, the computational complexity of empirical JSD depends on the size of the support or samples and dimensionality. It involves computing two KLD terms.

4 Maximum Mean Discrepancy (MMD)

Maximum Mean Discrepancy (MMD) is a statistical measure used to evaluate the discrepancy between two probability distributions. It is foundational in tasks like domain adaptation, hypothesis testing, and independence assessment. MMD is defined as the largest mean difference of functions in a Reproducing Kernel Hilbert Space (RKHS), serving as an integral probability metric that captures differences in distributions by comparing their mean embeddings in the RKHS [56].

Formally, for two distributions p and q over a domain $\mathcal{X}$, MMD is expressed as:

$$\text{MMD}[\mathcal{H}, p, q] := \sup_{f \in \mathcal{H}, \|f\|_{\mathcal{H}} \leq 1} (\mathbb{E}_{x \sim p}[f(x)] - \mathbb{E}_{x \sim q}[f(x)]),$$

where $\mathcal{H}$ is the RKHS associated with a kernel function $k(x, x')$. The population MMD can be further simplified to the squared distance between the mean embeddings of p and q :

$$\text{MMD}^2[\mathcal{H}, p, q] = \|\mu_p - \mu_q\|_{\mathcal{H}}^2,$$

where $\mu_p = \mathbb{E}_{x \sim p}[k(x, \cdot)]$ and $\mu_q = \mathbb{E}_{x \sim q}[k(x, \cdot)]$ are the mean embeddings of p and q in $\mathcal{H}$.

For practical computation with finite samples, given data $X = \{x_1, \dots, x_m\}$ from p and $Y = \{y_1, \dots, y_n\}$ from q , the unbiased empirical estimate of MMD

is:

$$\text{MMD}_u^2 = \frac{1}{m(m-1)} \sum_{i \neq j} k(x_i, x_j) + \frac{1}{n(n-1)} \sum_{i \neq j} k(y_i, y_j) - \frac{2}{mn} \sum_{i=1}^m \sum_{j=1}^n k(x_i, y_j),$$

where k is a positive-definite kernel, such as the Gaussian or Laplacian kernel. These kernels are commonly used due to their characteristic property, ensuring injectivity of the mean embedding mapping [57].

A key advantage of MMD lies in its ability to capture both mean and higher-order differences in distributions when an appropriate kernel is chosen. This makes MMD particularly useful in non-parametric two-sample testing and domain adaptation tasks, where aligning distributions in a shared latent space is critical. However, computational cost, which scales quadratically with the sample size, can be a limitation. Techniques like linear-time approximations and kernel selection strategies help mitigate this challenge [56].

MMD has been extensively applied in domain adaptation to align source and target domain feature distributions in a latent space, facilitating the transfer of learning [58]. By minimizing MMD during training, models are encouraged to learn domain-invariant representations, thereby improving generalization across different but related domains.

5 Central Moment Discrepancy (CMD)

Domain adaptation addresses the challenge of applying machine learning models trained on a source domain to a different but related target domain, particularly when labeled data is scarce or unavailable in the target domain. A common and effective strategy, especially within deep learning frameworks, is to learn domain-invariant representations [21]. This involves transforming the input data from both source and target domains into a shared latent feature space

where the distributions are aligned, thereby allowing a model trained on source data to generalize better to the target domain.

Several methods have been proposed to measure and minimize the discrepancy between domain distributions in the latent space. Maximum Mean Discrepancy (MMD), often employing kernels like the Gaussian kernel, measures distance in a Reproducing Kernel Hilbert Space (RKHS). While effective, MMD can be interpreted as matching a weighted sum of all moments, requires potentially costly kernel matrix computations, and necessitates careful tuning of kernel hyperparameters. Other approaches, like minimizing the Kullback-Leibler (KL) divergence between the means of activations (MKL), focus only on the first moment, potentially missing finer distributional differences captured by higher-order statistics. Domain-adversarial training (DANN) introduces an auxiliary domain classifier and an adversarial objective, adding complexity through extra parameters and training dynamics.

To address the limitations of existing methods, Zellinger et al. [21] introduced the Central Moment Discrepancy (CMD) as a novel metric specifically designed for learning domain-invariant representations in neural networks. CMD explicitly matches the moments of the domain-specific activation distributions order-by-order. Unlike MMD’s implicit weighting of moments or MKL’s focus solely on the mean, CMD aligns higher-order moments by minimizing the difference between corresponding central moments of the source and target distributions up to a specified order. Central moments are chosen due to their translation invariance and their intuitive geometric interpretation, relating to variance, skewness, kurtosis, and higher-order moments.

Formally, for two probability distributions p and q on a compact interval $[a, b]^N$, represented by random vectors $X \sim p$ and $Y \sim q$, the CMD metric is

defined as:

$$\text{CMD}(p, q) = \frac{1}{|b - a|} \|\mathbb{E}(X) - \mathbb{E}(Y)\|_2 + \sum_{k=2}^{\infty} \frac{1}{|b - a|^k} \|c_k(X) - c_k(Y)\|_2,$$

where $\mathbb{E}(\cdot)$ denotes the expectation, $\|\cdot\|_2$ is the L2 norm, and $c_k(X)$ represents the vector of all k -th order central moments of X . CMD is a valid metric on the space of probability distributions over a compact interval, and minimizing CMD ensures convergence in distribution.

For computational efficiency, an empirical estimate CMD_K is used:

$$\text{CMD}_K(X, Y) = \frac{1}{|b - a|} \|\mathbb{E}(X) - \mathbb{E}(Y)\|_2 + \sum_{k=2}^K \frac{1}{|b - a|^k} \|C_k(X) - C_k(Y)\|_2,$$

where X and Y are samples (e.g., mini-batches of hidden activations from source and target domains), and K is a hyperparameter determining the highest order moment to match. CMD's empirical version approximates the infinite sum and uses marginal moments for computational tractability, reducing complexity while maintaining alignment accuracy.

The primary advantages of CMD include:

- **Explicit Higher-Order Matching:** Directly aligns distributions based on higher-order statistics beyond just the mean or variance.
- **Computational Efficiency:** The empirical estimate CMD_K (using marginal moments) has a computational complexity linear in the number of samples (n, m) and the feature dimension (N) , specifically $\mathcal{O}(NK(n+m))$, which is significantly more efficient than the $\mathcal{O}(N(n^2 + nm + m^2))$ complexity of standard kernel MMD.
- **No Kernel Computation:** Avoids the need for kernel matrix calculations and the associated hyperparameter tuning (e.g., kernel bandwidth).

- **Stability:** Empirical results suggest CMD is relatively insensitive to the choice of K for $K \geq 3$, reducing the burden of hyperparameter optimization.

CMD is incorporated into the training process by adding it as a regularization term to the neural network’s loss function. This encourages the network to learn latent representations that are domain-invariant while maintaining performance on the primary task. Empirical evaluations by Zellinger et al[21]. showed that CMD achieves state-of-the-art results on standard domain adaptation benchmarks.

Chapter 4

Conformal Prediction

Conformal Prediction (CP) is a framework for uncertainty quantification that provides prediction sets or intervals with a guaranteed coverage rate under the assumption of data exchangeability. The framework ensures that these sets contain the true label with a user-specified confidence level $1 - \alpha$, under the assumption of data exchangeability [59]. CP operates in three main steps: computation of non-conformity scores, quantile computation, and construction of prediction sets or intervals. This methodology provides a rigorous way to quantify uncertainty in predictions.

Non-conformity Scores : Non-conformity scores quantify how well a given label Y aligns with the predictions made for an input X . For regression tasks, a typical non-conformity score is the residual, defined as $V(X, Y) = |Y - \hat{Y}(X)|$, where $\hat{Y}(X)$ is the model's prediction for X . In classification tasks, these scores can be based on class probabilities or rankings. For example, Adaptive Prediction Sets (APS) use cumulative class probabilities to define non-conformity scores, as described later in this section.

Quantile Computation : The second step involves determining a threshold that ensures the desired coverage. Using a calibration dataset $\{(X_i, Y_i)\}_{i=1}^n$, the threshold τ is computed as the $(1-\alpha)(1+1/n)$ -th quantile of the non-conformity scores:

$$\tau = \text{quantile} \left(V(X_1, Y_1), \dots, V(X_n, Y_n), (1 - \alpha)(1 + \frac{1}{n}) \right).$$

This threshold represents the score below which the top $(1 - \alpha)$ -fraction of the calibration samples fall, ensuring the desired coverage.

Prediction Set or Interval Construction For a new test input X_{n+1} , CP constructs a prediction set $C(X_{n+1})$ based on the non-conformity score and the computed threshold:

$$C(X_{n+1}) = \{Y \in \mathcal{Y} : V(X_{n+1}, Y) \leq \tau\}.$$

This prediction set contains all labels Y that are considered plausible given the score V and the threshold τ . The framework ensures that, under the assumption of exchangeability, the true label is contained in $C(X_{n+1})$ with probability at least $1 - \alpha$.

Adaptive Prediction Sets (APS) : For classification tasks, APS employs a specific non-conformity score based on cumulative class probabilities. Let $\hat{P}(Y = j|X)$ denote the predicted probability of class j . The score is defined as:

$$V(X, k) = \sum_{j=1}^k \hat{P}(Y = j|X),$$

where the classes are ranked in descending order of their probabilities. The prediction set $C(X)$ includes the top k classes until their cumulative probability

exceeds τ . This approach ensures valid coverage while adapting the size of the prediction set to the model’s confidence.

Evaluation Metrics : Conformal Prediction aims to achieve valid marginal coverage while minimizing inefficiency. The marginal coverage for a test dataset $\mathcal{D}_{\text{test}}$ is defined as:

$$\text{Coverage} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{i \in \mathcal{D}_{\text{test}}} \mathbb{I}(Y_i \in C(X_i)),$$

where $\mathbb{I}(\cdot)$ is the indicator function. Inefficiency measures the size of prediction sets. For classification tasks, it is computed as:

$$\text{Inefficiency} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{i \in \mathcal{D}_{\text{test}}} |C(X_i)|.$$

For regression, inefficiency corresponds to the average length of the prediction intervals. The goal is to balance valid coverage with small, efficient prediction sets.

Exchangeability and Validity : To apply CP to graph-structured data, exchangeability of node information must be preserved under the transductive setting. Under a general permutation invariance condition, the exchangeability of non-conformity scores remains valid even when GNN training involves calibration and test information [59]. This ensures that CP can be reliably used for GNN models. For detailed proofs and theoretical results, refer to [59].

Chapter 5

Methodology

Graph Neural Networks (GNNs) have demonstrated remarkable performance in node classification tasks. However, their effectiveness can be compromised when the training data distribution differs from the test data distribution, a phenomenon known as distributional shift. In this work, we address the conditional shift problem in GNNs, where the distribution of node embeddings in the training set $P(Z_\ell)$ differs from the distribution in the full dataset $P(Z)$. Let $G = (V, E)$ be a graph with n nodes, where V represents the set of nodes and E represents the set of edges. Each node $i \in V$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^d$ and a label $y_i \in 1, 2, \dots, C$, where C is the number of classes. The adjacency matrix of the graph is denoted as $\mathbf{A} \in \mathbb{R}^{n \times n}$. Let $Z = z_1, z_2, \dots, z_n$ represent the node embeddings from the final hidden layer of a GNN applied to graph G . Given labeled data $(\mathbf{x}_i, y_i)_{i=1}^m$ of size m , the labeled node embeddings $Z_\ell = z_1, \dots, z_m$ form a subset of the node embeddings that are labeled, with $Z_\ell \subseteq Z$. We assume Z and Z_ℓ are drawn from two probability distributions P and Q , respectively. The conditional shift in GNNs is measured using a distance metric $d(Z, Z_\ell)$.

0.1 Graph Neural Network Framework

Consider a conventional GNN model Ψ with a learnable function g with parameters η and adjacency matrix $\mathbf{A}$:

$$\Psi = g(\eta, \mathbf{A}, \mathbf{Z}),$$

where $\mathbf{Z} := \mathbf{Z}_L = g(\eta, \mathbf{A}, \mathbf{Z}_{L-1})$ and $\mathbf{Z}_0 = \mathbf{X}$. We have $\mathbf{Z}_\ell \in [\alpha, \beta]^n$ since a bounded activation function is employed. Let the training samples from the dataset $\tilde{\mathcal{D}}_{\text{train}}$ be denoted as $\{x_i\}_{i=1}^m$, with the corresponding node embeddings represented as $\mathbf{Z}_{\text{train}} = \{z_i\}_{i=1}^m$. For the test samples, we take an unbiased IID sample from the unlabeled data $\mathbf{X}_{\text{IID}} = \{x'_i\}_{i=1}^m$ and denote the output embeddings as $\mathbf{Z}_{\text{IID}} = \{z'_i\}_{i=1}^m$.

0.2 Regularization

To mitigate the distributional shift between training and testing, we introduce a regularizer $d : [\alpha, \beta]^n \times [\alpha, \beta]^n \rightarrow \mathbb{R}^+$, which is added to the cross-entropy loss. This regularizer quantifies the discrepancy between the predicted labels $\hat{y}_i$ generated by the GNN for each node i and the true labels y_i . The total loss $\mathcal{L}$ is computed as the average of the individual losses across all training instances m . Since Ψ is fully differentiable, we can incorporate the distributional shift metric as a regularization term to directly reduce the discrepancy between the biased training sample and the unbiased IID sample, formulated as:

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^m \ell(y_i, \hat{y}_i) + d(\mathbf{Z}_{\text{train}}, \mathbf{Z}_{\text{IID}}),$$

where $\ell(y_i, \hat{y}_i)$ represents the loss between the true and predicted labels.

0.3 Our Proposed Regularization Framework

We take the regularization term $d(\cdot)$ as a linear combination of different metrics. Let $g_1, g_2, \dots, g_J$ represent different metric functions like CMD, MMD, KLD, etc. The regularizer can then be expressed as a weighted sum of these metrics:

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^m \ell(y_i, \hat{y}_i) + d(\mathbf{Z}_{\text{train}}, \mathbf{Z}_{\text{IID}}),$$

where,

$$d(\mathbf{Z}_{\text{train}}, \mathbf{Z}_{\text{IID}}) = \sum_{j=1}^J \lambda_j \cdot g_j(\mathbf{Z}_{\text{train}}, \mathbf{Z}_{\text{IID}}).$$

The regularizer $d(\mathbf{Z}_{\text{train}}, \mathbf{Z}_{\text{IID}})$ is defined as a combination of different metrics $g_1, g_2, \dots, g_J$. Each metric $g_j(\mathbf{Z}_{\text{train}}, \mathbf{Z}_{\text{IID}})$ contributes to the overall regularization term, and the coefficients α_j determine the weight of each metric in the combination.

For example, if g_1 as KLD, g_2 as MMD and g_3 as CMD, then the regularizer can be written as:

$$d(\mathbf{Z}_{\text{train}}, \mathbf{Z}_{\text{IID}}) = \lambda_1 \cdot d_{\text{KLD}}(\mathbf{Z}_{\text{train}}, \mathbf{Z}_{\text{IID}}) + \lambda_2 \cdot d_{\text{MMD}}(\mathbf{Z}_{\text{train}}, \mathbf{Z}_{\text{IID}}) + \lambda_3 \cdot d_{\text{CMD}}(\mathbf{Z}_{\text{train}}, \mathbf{Z}_{\text{IID}}).$$

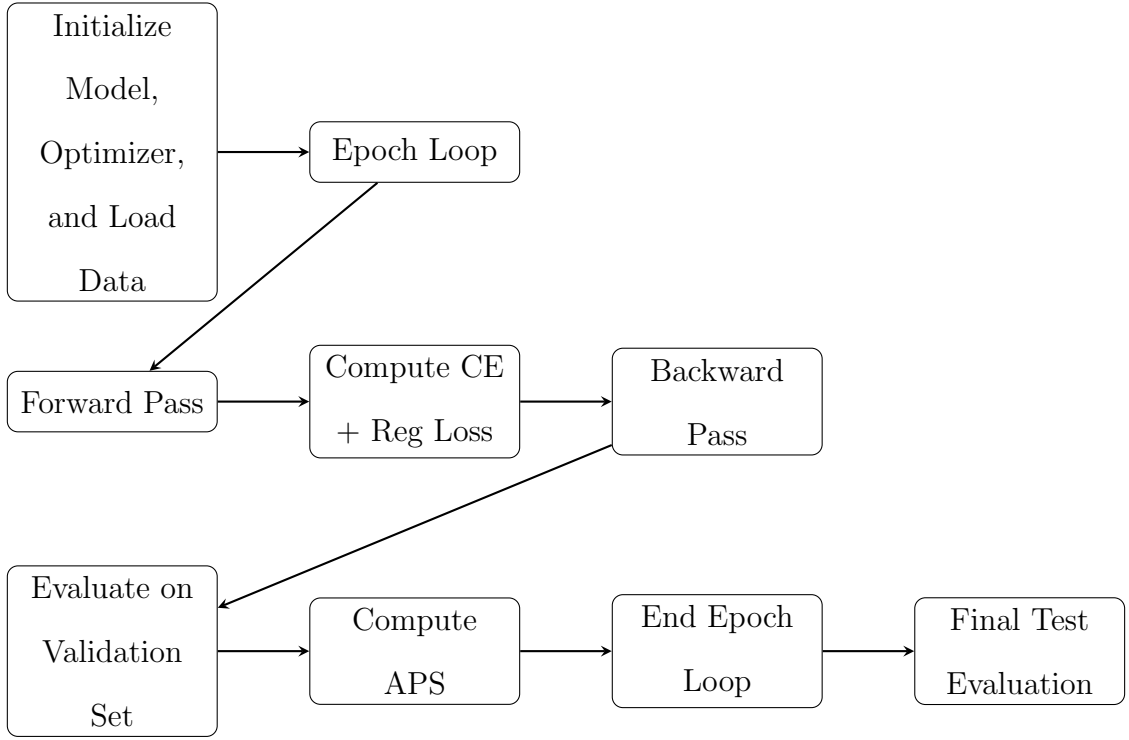
This formulation helps to combine the effectiveness of different metrics in a single regularization term, making the loss function more robust and adaptive.

In practice, adjusting the relative weights of each regularizer by tuning the coefficients α_j controls whether the model prioritizes accurate class confidence, structural consistency in learned representations, or overall distributional robustness. For instance, when the primary task loss (such as Cross-Entropy, CE) is combined mainly with a KLD term ($\lambda_{\text{KLD}} > 0$, $\lambda_{\text{CMD}} = \lambda_{\text{MMD}} = 0$), the model is guided to maintain well-aligned class probabilities at every node, reinforcing reliable prediction confidence by minimizing the divergence from

the target distribution. In contrast, enabling Central Moment Discrepancy (CMD) and MMD together ($\lambda_{\text{CMD}}, \lambda_{\text{MMD}} > 0$, $\lambda_{\text{KLD}} = 0$) shifts learning toward matching both local feature statistics and global embedding patterns of the learned representations: CMD enforces similarity in the central moments (including mean, variance, and higher orders) of representations in different neighborhoods or domains, while MMD drives the overall feature distributions in a Reproducing Kernel Hilbert Space (RKHS) to coincide, enhancing stability, for example, across shifts in graph structure. A hybrid setting ($\lambda_{\text{CMD}}, \lambda_{\text{KLD}} > 0$, $\lambda_{\text{MMD}} = 0$) balances these aims, utilizing CMD to regularize the structural properties captured by moment statistics and KLD to keep the network’s probabilistic outputs properly measured by reducing distribution divergence. Through the adjustment of the λ_j coefficients, the GNN can be precisely tuned to prioritize these different aspects—prediction confidence (linked to KLD), the alignment of statistical moments (governed by CMD), and overall distributional similarity (achieved with MMD, if active)—thereby tailoring the model effectively to the task’s specific structural and statistical requirements.

0.4 Training Procedure

The training process for the GNN models, potentially incorporating the described regularization terms and size loss, follows a standard iterative optimization procedure. The overall process for a single experimental run is illustrated below:



We conduct experiments on benchmark graph datasets Cora, Citeseer. The models are implemented using PyTorch and Deep Graph Library (DGL). For each dataset, we compare different GNN architectures (GCN, GAT, GraphSAGE, APPNP, DAGNN) and different regularization strategies. The main performance metrics are:

Accuracy: F1(micro)-score for node classification. Coverage: The fraction of test instances for which the true label is included in the prediction set. Efficiency: The average size of the prediction sets (smaller is better, given the same coverage).

0.5 Configuration

Based on our hyperparameter tuning experiments, we found that the optimal configuration varies depending on the dataset and the specific task. However, some general trends emerged:

- **Learning rate:** Values around 0.01 typically yielded the best performance.
- **Regularization weights:** λ_1 around 0.7 and λ_2 around 0.1 worked well for most datasets.
- **Hidden dimension:** 32 to 64 provided a good trade-off between model capacity and computational efficiency.
- **Number of layers:** 2 layers were sufficient for most datasets, with deeper models sometimes overfitting.
- **Dropout rate:** 0.5 was effective in preventing overfitting.
- **Propagation steps:** 12 for Cora and Citeseer.
- **Biasing parameter:** 0.1 introduced a moderate level of bias without completely distorting the training distribution.
- **Conformal significance level:** 0.1 for the main experiments.

These settings provided a good balance between accuracy, coverage guarantee satisfaction, and efficiency of the prediction sets. However, we emphasize that the optimal configuration is highly dependent on the specific dataset and task, and a systematic hyperparameter tuning should be performed for each new application.

Chapter 6

Experiments

In this segment, we evaluate our proposed methodology for quantifying uncertainty when faced with conditional shift in data. Our experiment focuses, where the Graph Neural Network is trained on a localized subset drawn from a distribution that differs from the test data. We assess our proposition’s performance by applying it to the baseline models. Recognizing the impact of hyperparameters on the effectiveness of our technique we did optimisation of these parameters to get the optimal outcome and did an analysis (refer to Ablation study).

0.1 Biased Training Set Creation

To examine conditional shifts in Graph Neural Networks (GNNs), it is crucial to have a iterative process for generating biased training datasets that reflect localized patterns within the graph. Such datasets help assess the robustness of GNNs under non-IID training conditions. In this work, biased data is generated by leveraging Personalized PageRank (PPR) vectors [44], which effectively capture the proximity relationships in graphs. These vectors are defined as:

$$\mathbf{ppr} = (\mathbf{I} - (1 - \alpha)\tilde{\mathbf{A}})^{-1},$$

where $\tilde{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}}(\mathbf{A} + \mathbf{I})\mathbf{D}^{-\frac{1}{2}}$ represents the normalized adjacency matrix, $\mathbf{A} + \mathbf{I}$ includes self-loops, and $\mathbf{D}$ is the degree matrix.

The parameter $\alpha \in [0, 1]$ governs the locality of the PPR vector: smaller values of α emphasize nodes that are closer in the graph, generating highly localized samples, while $\alpha = 1$ reduces the vector to the normalized adjacency matrix, resulting in more global node selection. PPR vectors are computationally efficient, enabling their calculation for individual nodes in sublinear time relative to the graph size [60]. This property makes the approach scalable for large datasets, where stable and efficient approximations of the top entries in the PPR vector can be computed with guaranteed truncation error.

The methodology for constructing biased datasets involves the following steps, follows from [61]. For a specified label distribution ratio, the number of training nodes required for each class is determined in advance. Seed nodes are selected randomly, ensuring their PPR vectors contain sufficient neighbors with the same label. Both the seed nodes and their labeled neighbors are iteratively added to the training data until the desired number of nodes per label is obtained. This process ensures that the training set reflects the intended bias while maintaining control over the graph structure. The biased training sets generated in this way effectively challenge the GNN to adapt to scenarios with localised, unevenly distributed data.

0.2 Datasets :

In our experiment, we performed semi-supervised node classification task using three datasets: Cora, Citeseer. To generate the biased training data we followed the data generation process in Section II and generated biased datasets of $\alpha = 0.1$, i.e., the training set is 90% biased and as the α increases the training data becomes more similar to test data. For the calibration data we

allocate a subset of data with a size equal to the minimum of 1000 or half of the combined samples in calibration and test dataset, i.e., $\min \left\{ 1000, \frac{|D_{\text{calib}} \cup D_{\text{test}}|}{2} \right\}$. The remaining are test set under the assumption that both are drawn from identical and independent distributions(IID).

0.3 Models and Implementation :

In our experiment, we used two categories of GNN models. The first category consists of traditional GNNs, GCN, GAT, GraphSage, which integrate message passing and transformation. The second category models separate these process, APPNP and DAGNN. We train these models on our generated biased data with our proposed regularizatio terms which leverages different combinations of various distance metrics. Then we apply Conformal Prediction (CP) to quantify uncertainty and generate prediction sets. Then we evaluate accuracy, marginal coverage and prediction set size for different datasets Cora, Citeseer with the biasing setting of $\alpha = 0.1$. To adjust the parameters α_j 's further fine-tuning was done to enhance accuracy and to get a much tighter prediction set while maintaining the 90% coverage.

Chapter 7

Results

0.1 Accuracy Comparison

Table 7.1: Comparison of Accuracy for Cora Dataset

Model	IID	CE	CJS	CKL	MMD	SRGNN	CMD	JSD	KLD
APPNP	0.8562	0.7101	0.7278 $\pm$ 0.02	0.7318 $\pm$ 0.02	0.7175 $\pm$ 0.01	0.7530 $\pm$ 0.01	0.7304 $\pm$ 0.01	0.7094 $\pm$ 0.02	0.7202 $\pm$ 0.01
DAGNN	0.8229	0.7179	0.7232 $\pm$ 0.01	0.7345 $\pm$ 0.01	0.7187 $\pm$ 0.002	0.7312 $\pm$ 0.01	0.7305 $\pm$ 0.01	0.7173 $\pm$ 0.002	0.7175 $\pm$ 0.003
GAT	0.808	0.6861	0.6771 $\pm$ 0.03	0.6847 $\pm$ 0.02	0.6801 $\pm$ 0.01	0.6798 $\pm$ 0.03	0.6773 $\pm$ 0.03	0.6905 $\pm$ 0.01	0.6846 $\pm$ 0.009
GCN	0.7991	0.6675	0.6780 $\pm$ 0.004	0.6812 $\pm$ 0.005	0.6672 $\pm$ 0.006	0.6801 $\pm$ 0.007	0.6758 $\pm$ 0.007	0.6663 $\pm$ 0.005	0.6680 $\pm$ 0.007
GraphSAGE	0.8084	0.6924	0.7344 $\pm$ 0.01	0.7282 $\pm$ 0.01	0.6888 $\pm$ 0.009	0.7333 $\pm$ 0.01	0.7357 $\pm$ 0.01	0.6913 $\pm$ 0.01	0.6878 $\pm$ 0.006

Table 6.1 showcases the accuracy results for the Cora dataset, averaged over 15 runs for five GNN models—APPNP, DAGNN, GAT, GCN, and GraphSAGE, under conditional shift with a bias of $\alpha = 0.1$ across various metrics. APPNP achieves the highest accuracy on IID data (0.8562), showcasing its robustness under in-distribution conditions. Under biased scenarios, both APPNP and GraphSAGE demonstrate strong adaptability across different metrics. APPNP leads in CKL (0.7318, 0.5% higher than GraphSAGE) and SRGNN (0.7530, 2.7% higher than GraphSAGE), while GraphSAGE achieves superior performance in CE (0.7344, 0.9% higher than APPNP) and CMD (0.7357, 0.7% higher than APPNP). These results highlight that both models are well-suited to handle conditional shifts, with APPNP excelling in certain metrics and GraphSAGE

demonstrating consistent adaptability across others.

Table 7.2: Comparison of Accuracy for Citeseer Dataset

Model	IID	CE	CJS	CKL	MMD	SRGNN	CMD	JSD	KLD
APNP	0.6901	0.6071	0.6158 $\pm$ 0.0188	0.6156 $\pm$ 0.0255	0.5937 $\pm$ 0.0164	0.6257 $\pm$ 0.0227	0.6180 $\pm$ 0.0243	0.6079 $\pm$ 0.0204	0.6190 $\pm$ 0.0231
DAGNN	0.6713	0.6019	0.6027 $\pm$ 0.0308	0.6091 $\pm$ 0.0219	0.5882 $\pm$ 0.0326	0.6103 $\pm$ 0.0219	0.6217 $\pm$ 0.0118	0.6061 $\pm$ 0.0039	0.5918 $\pm$ 0.0287
GAT	0.6901	0.5948	0.5799 $\pm$ 0.0470	0.5789 $\pm$ 0.0510	0.5968 $\pm$ 0.0102	0.5747 $\pm$ 0.0572	0.5608 $\pm$ 0.0371	0.6041 $\pm$ 0.0222	0.6002 $\pm$ 0.0194
GCN	0.7030	0.5921	0.5913 $\pm$ 0.0064	0.5939 $\pm$ 0.0063	0.5881 $\pm$ 0.0062	0.5911 $\pm$ 0.0066	0.5915 $\pm$ 0.0060	0.5897 $\pm$ 0.0059	0.5874 $\pm$ 0.0070
GraphSAGE	0.6971	0.6124	0.6374 $\pm$ 0.0125	0.6374 $\pm$ 0.0126	0.6008 $\pm$ 0.0130	0.6360 $\pm$ 0.0092	0.6377 $\pm$ 0.0117	0.5990 $\pm$ 0.0114	0.6049 $\pm$ 0.0136

Table 6.2 showcases the average and standard deviation of accuracy(F_1 micro has been used for measuring accuracy), based on 15 GNN runs of the five GNN models—APNP, DAGNN, GAT, GCN, and GraphSAGE on the Citeseer dataset across multiple combinations of metrics the results represent . GCN achieves the highest IID accuracy (0.703), but it exhibits the most significant decline under biased conditions, with a reduction of 15.8%. In contrast, GraphSAGE demonstrates notable improvement and adaptability, achieving the best performance across several metrics: Cross-Entropy with Confidence Interval Shift (CE, 4.4% higher than APNP), CKL (3.5% better than GAT), Central Moment Discrepancy (CMD, 3.2% higher than DAGNN), and SRGNN (2.6% better than APNP). APNP, while slightly behind in some metrics, leads in Kullback-Leibler Divergence (KLD, 3.4% higher than DAGNN). These results underscore the varying strengths of GNN models under different combinations of metrics, and GraphSAGE shows better performance in handling conditional shifts.

0.2 Coverage Results

Our comprehensive evaluation of coverage for different metrics across different graph neural network architectures reveals significant insights regarding their generalization capabilities on the Cora and Citeseer datasets, with all models exceeding the coverage threshold of 90% for unbiased data(IID) and so as most

Table 7.3: Comparison of Coverage for Cora Dataset

Model	IID	CE	CJS	CKL	MMD	SRGNN	CMD	JSD	KLD
APPNP	0.9015	0.9016	0.8992	0.8995	0.9015	0.8993	0.8990	0.9016	0.9015
DAGNN	0.9011	0.9019	0.9002	0.9003	0.9018	0.9006	0.9004	0.9019	0.9018
GAT	0.9025	0.9016	0.8992	0.8993	0.9017	0.8993	0.8994	0.9017	0.9015
GCN	0.9005	0.9026	0.9027	0.9028	0.9025	0.9028	0.9027	0.9027	0.9027
GraphSAGE	0.9030	0.9017	0.8990	0.8990	0.9015	0.8992	0.8992	0.9016	0.9015

of the proposed methods under the presence of bias, which indicates strong prediction reliability. On the Cora dataset, GAT demonstrates maximum IID coverage (0.9025), outperforming GCN (0.9005), APPNP (0.9015,), DAGNN (0.9011), and GraphSAGE (0.903). Under biased distribution scenarios, GCN exhibits the highest coverage (0.9026), while GraphSAGE shows the most substantial decline (0.9017 from its IID performance). Under conditional shift taking Cross-Entropy(CE) as only loss function, GCN achieves the highest performance (0.9027), followed by DAGNN (0.9002). For CKL(CMD+KLD), GCN maintains coverage (0.9028), with GraphSAGE showing the lowest performance (0.8990). Maximum Mean Discrepancy (MMD) evaluation reveals GCN (0.9025) and GraphSAGE (0.9015) as the most and least effective models, respectively. For SRGNN(CMD+MMD), GCN again demonstrates optimal performance (0.9028). Across divergence-based metrics, GCN consistently outperforms other architectures with Central Moment Discrepancy (CMD) of 0.9027, Jensen-Shannon Divergence (JSD) of 0.9027, and Kullback-Leibler Divergence (KLD) of 0.9027.

Table 7.4: Comparison of Coverage for Citeseer Dataset

Model	IID	CE	CJS	CKL	MMD	SRGNN	CMD	JSD	KLD
APNP	0.9007	0.9022	0.9020	0.9019	0.9023	0.9025	0.9020	0.9025	0.9018
DAGNN	0.9020	0.9029	0.9021	0.9019	0.9024	0.9017	0.9021	0.9030	0.9029
GAT	0.9011	0.9013	0.9021	0.9023	0.9015	0.9019	0.9019	0.9016	0.9016
GCN	0.9009	0.9031	0.9029	0.9030	0.9033	0.9028	0.9030	0.9031	0.9031
GraphSAGE	0.9004	0.9015	0.9018	0.9016	0.9014	0.9022	0.9022	0.9015	0.9015

In **Table 6.4** For the Citeseer dataset, DAGNN exhibits marginally higher IID coverage (0.902) compared to GAT (0.9011), GCN (0.9009), APPNP (0.9007), and GraphSAGE (0.9004). Under biased conditions, GCN demonstrates the highest coverage (0.9031), indicating superior robustness. For CE-CIS, GCN achieves the highest performance (0.9029), followed closely by DAGNN (0.9021). Under CKL evaluation, GCN maintains superior coverage (0.9030), while GraphSAGE exhibits the lowest (0.9016). Maximum Mean Discrepancy (MMD) reveals GCN (0.9033) as the most effective model in this case, with GraphSAGE (0.9014) demonstrating the lowest performance. For SRGNN, GCN again leads (0.9028), while GraphSAGE trails (0.9022). Across divergence-based metrics, GCN consistently outperforms other architectures with CMD (0.9030), JSD (0.9031), and KLD (0.9031).

These results demonstrate that while GAT and DAGNN exhibit marginally higher coverage for IID on Cora and Citeseer respectively, GCN consistently demonstrates significant improvement across different metrics for both unbiased and unbiased datasets. This suggests that GCN’s spectral-based approach may provide more stable coverage guarantees under varying distribution conditions than the attention mechanisms in GAT or the aggregation strategies in GraphSAGE. The minimal performance variations across models (generally less than 0.5%) indicate that all evaluated architectures maintain robust coverage well above the critical 90% threshold, representing strong predictive reliability across distribution shifts. Notably, GCN exhibits a slight but consistent advantage

when subjected to distribution shifts, suggesting particular resilience in real-world scenarios where distribution shifts are inevitable.

0.3 Prediction Set Size

Table 7.5: Comparison of Prediction Set Size for Cora Dataset

Model	IID	CE	CJS	CKL	MMD	SRGNN	CMD	JSD	KLD
APPNP	3.8159	4.1407	2.2595	2.3306	4.0549	2.3003	2.3256	4.1400	4.0595
DAGNN	4.5173	4.7468	3.2271	3.3076	4.7400	3.2751	3.2262	4.7476	4.7476
GAT	3.4103	3.9805	2.7659	2.6841	4.0488	2.7158	2.7569	3.9182	3.9774
GCN	3.8876	4.1933	4.1019	4.1097	4.2021	4.1070	4.1287	4.2187	4.2011
GraphSAGE	3.3313	3.6772	2.1970	2.2793	3.5620	2.2125	2.1734	3.5411	3.7013

On the Cora dataset, GraphSAGE consistently demonstrated superior precision by achieving the smallest prediction set sizes across unbiased and biased cases. Under IID settings, GraphSAGE yielded a prediction set size of 3.3113, outperforming GAT (3.4103), while APPNP (3.8159) and GCN (3.8876) recorded notably larger sets, and DAGNN (4.5173) exhibited the largest, indicating a considerable gap in precision. This trend of GraphSAGE leading in precision continued under biased data conditions, where its set size of 3.6772 was again smaller than GAT’s 3.9805. Furthermore, GraphSAGE’s dominance extended across all metrics. For instance, in CJS, its set size was 2.1970, and in CKL, it was 2.2793, in both cases being more precise than APPNP, which recorded 2.2595 and 2.3306 respectively. For MMD, GraphSAGE (3.5620) was more precise than GAT (4.0488). Similarly, for SRGNN, GraphSAGE’s set size of 2.2125 was smaller than APPNP’s 2.3003. But across the divergence-based metrics, GraphSAGE also consistently achieved the smallest prediction sets: 2.1734 for cmd, 3.5411 for jsd, and 3.7013 for kld. This consistent outperformance signifies that GraphSAGE offers the highest prediction precision for the Cora dataset under a variety of data distributions, while models like DAGNN and GCN generally produced substantially larger prediction set sizes, indicating lower precision in this context.

Table 7.6: Comparison of Prediction Set Size for Citeseer Dataset

Model	IID Predict	CE	CJS	CKL	MMD	SRGNN	CMD	JSD	KLD
APPNP	3.1048	3.1934	2.9465	2.9300	3.2363	2.9716	2.9452	3.2198	3.1422
DAGNN	3.7402	3.8453	3.4573	3.5891	3.8994	3.5508	3.3835	3.8361	3.9019
GAT	2.8967	3.0972	2.4854	3.6121	3.1458	3.4657	3.5596	3.1140	3.1732
GCN	3.0117	3.5631	3.5542	3.5652	3.5776	3.5787	3.5803	3.6004	3.5993
GraphSAGE	2.8921	3.0844	3.0844	3.0555	3.1197	3.0724	3.0641	3.1152	3.0839

The performance landscape for prediction set size on the Citeseer dataset is more varied compared to Cora, with several models demonstrating competitive precision depending on the specific metrics. Under IID conditions, GraphSAGE achieved the smallest prediction set size at 2.8921, marginally better than GAT (2.8967). Other models like GCN (3.0117) and APPNP (3.1048) had slightly larger sets, while DAGNN (3.7402) again showed the largest set size. In biased conditions, GraphSAGE (3.0844) maintained a slight edge in precision, closely followed by GAT (3.0972) and APPNP (3.1934). For several distribution shift metrics, APPNP emerged as a strong performer, achieving the smallest prediction set sizes for CE CJS (2.9465), CKL (2.9300), SRGNN (2.9716), and cmd (2.9452). For instance, its CE CJS value of 2.9465 was notably smaller than that of other models like GCN (3.5542). GraphSAGE also demonstrated high precision in specific shift scenarios, recording the smallest set size for MMD (3.1197), where it was more precise than GAT (3.1458), and for kld (3.0839). GAT, in turn, was the most precise for the jsd metric with a set size of 3.1140, performing almost identically to GraphSAGE (3.1152). In contrast, GCN and DAGNN generally exhibited larger prediction set sizes across most metrics on Citeseer. Thus, for the Citeseer dataset, GraphSAGE, APPNP, and GAT each demonstrated strengths in achieving smaller prediction set sizes under different conditions, indicating a more nuanced picture of precision leadership.

Chapter 8

Ablation Study

Here in this ablation study explores various metrics and their relationships using the Cora and Citeseer datasets. Below, plots and descriptions highlight key findings.

1 Cora

1.1 Accuracy vs. Methods

The presented bar plot illustrates the impact of different regularization methods, each employing various distance metrics, on the precision of five graph neural network architectures (APPNP, DAGNN, GAT, GCN and GraphSAGE) for the Cora dataset. This analysis directly ties into our broader exploration of handling conditional shifts in GNNs using different metrics.

The results clearly show that in the absence of conditional shift, the models achieve the highest accuracy. Meanwhile, methods such as CMD and MMD exhibit consistent performance, reflecting their capability to effectively minimize discrepancies and align feature distributions across shifts. CKL, combining CMD and KLD, achieves comparable accuracy, indicating the complementary

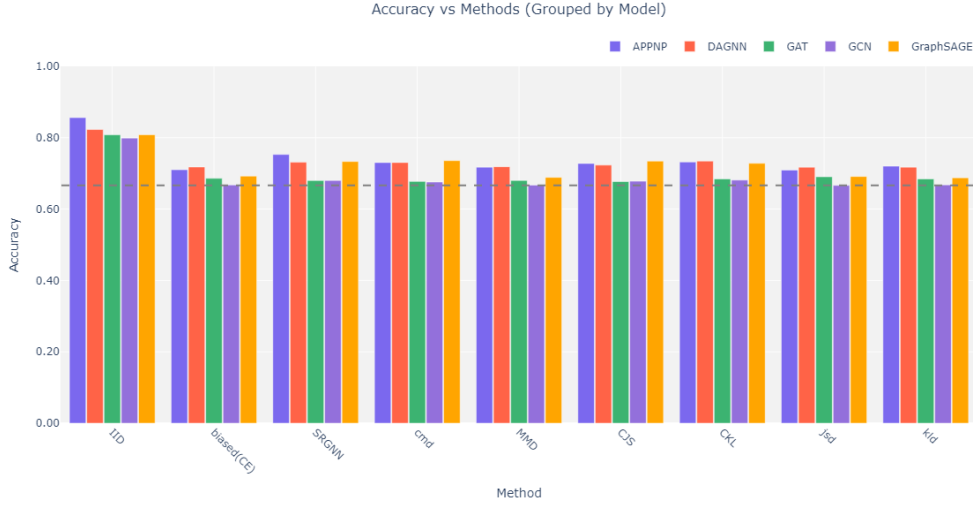


Figure 8.1: Comparative analysis of accuracy across different regularization methods for various graph neural network architectures on the Cora dataset.

strengths of these metrics in regularization.

Interestingly, biased(CE) [i.e., taking Cross Entropy(CE) as only regularizer to handle shifts] and JSD show slightly lower accuracies, which could be attributed to their sensitivity to over-regularization under specific architectural constraints. The tightly clustered results for most methods suggest that while some distance metrics excel under specific conditions, their overall performance differences are relatively small when applied to well-optimized GNNs.

APPNP and DAGNN models consistently outperform others and APPNP having the highest accuracy under SRGNN , demonstrating advantage of considering local and global structure of the data, suggesting their inherent design advantages in propagating information under conditional shifts. These findings highlight the critical role of selecting appropriate regularization strategies based on the dataset and the target distributional assumptions.

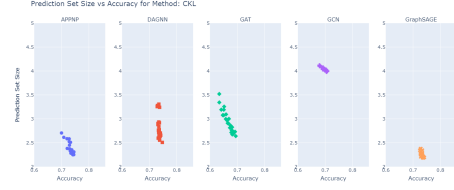
Above figure showcase towards the importance of leveraging combination of distance metrics to address conditional shifts effectively, aligning with the overarching goals of this thesis to enhance the robustness and adaptability of

GNNs in non-IID senarios.

1.2 Accuracy vs. Prediction Set Size (Fix Method, Vary Model)



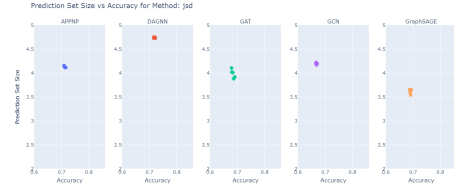
(a) CJS(CMD+JSD)



(b) CKL(CMD+KLD)



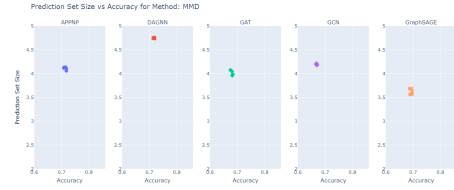
(c) CMD



(d) JSD



(e) KLD



(f) MMD



(g) SRGNN(CMD+MMD)

Figure 8.2: Accuracy vs. Prediction Set Size for fixed method and varying models on the Cora dataset.

The plots above represent how prediction set size influences accuracy when regularization methods are fixed and GNN architectures vary, focusing on

the Cora dataset. CMD-inclusive methods such as CJS, CKL, and SRGNN consistently demonstrate stable and high accuracy across different prediction set sizes, underscoring their ability to mitigate conditional shifts effectively. Among the architectures, APPNP and DAGNN exhibit superior performance relative to GCN and GAT, suggesting that these models are better suited to propagate structural information under regularization. Simpler metrics like KLD and JSD show greater sensitivity to prediction set size, with fluctuations in accuracy, reflecting their susceptibility to over-regularization. The findings also highlight that CMD-based combinations achieve a balance between accuracy and the informativeness of prediction sets, with DAGNN benefiting the most due to its enhanced local-global information propagation. These observations validate the efficacy of CMD integration within regularization frameworks and support the proposition of a combined approach to address conditional shifts.

1.3 Accuracy vs. Prediction Set Size (Fix Model, Vary Methods)

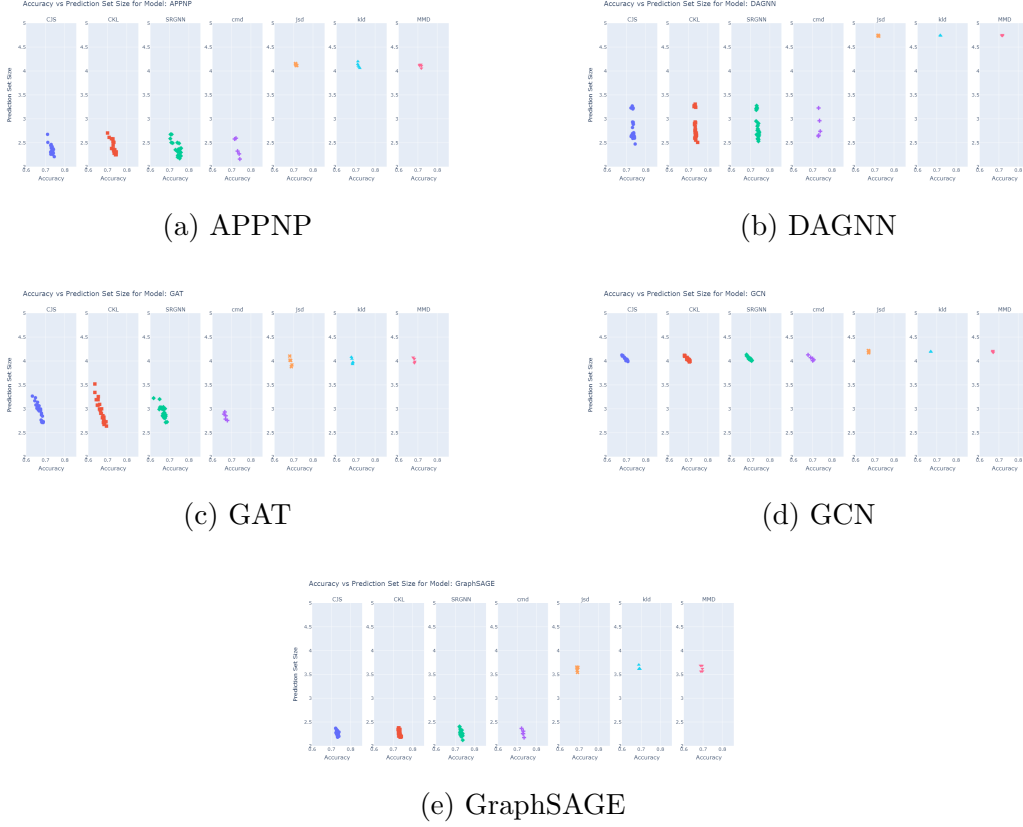


Figure 8.3: Accuracy vs. Prediction Set Size for fixed models with varying regularization methods on the Cora dataset.

Figure 8.3 illustrates the relationship between accuracy and prediction set size across five different GNN architectures (APPNP, DAGNN, GAT, GCN, and GraphSAGE) when various regularization methods are applied to the Cora dataset. The results reveal distinct patterns in how different regularization approaches impact the accuracy-size trade-off. CMD-based methods consistently demonstrate superior performance across architectures, maintaining high accuracy levels even as prediction set sizes vary. APPNP exhibits particularly remarkable stability with CMD and SRGNN regularization, where the accuracy remains elevated across the entire range of prediction set sizes, indicating robust feature

distribution alignment under conditional shifts. DAGNN similarly benefits from CMD-inclusive approaches, though with slightly more variability at larger prediction set sizes. In contrast, GAT and GCN display more pronounced sensitivity to the regularization method employed, with JSD and KLD showing steeper accuracy degradation as prediction set sizes increase. GraphSAGE presents an interesting middle ground, responding well to composite regularizers like CKL and SRGNN while showing less consistency with single-metric approaches. These observations underscore the differential impact of regularization strategies across architectural designs, with methods that effectively address distributional misalignments yielding more reliable performance. The plots clearly demonstrate that architectures with stronger local-global information propagation capabilities (APPNP and DAGNN) derive greater benefits from CMD-inclusive regularization frameworks when handling conditional shifts.

1.4 Method vs. Coverage (Fix Model)

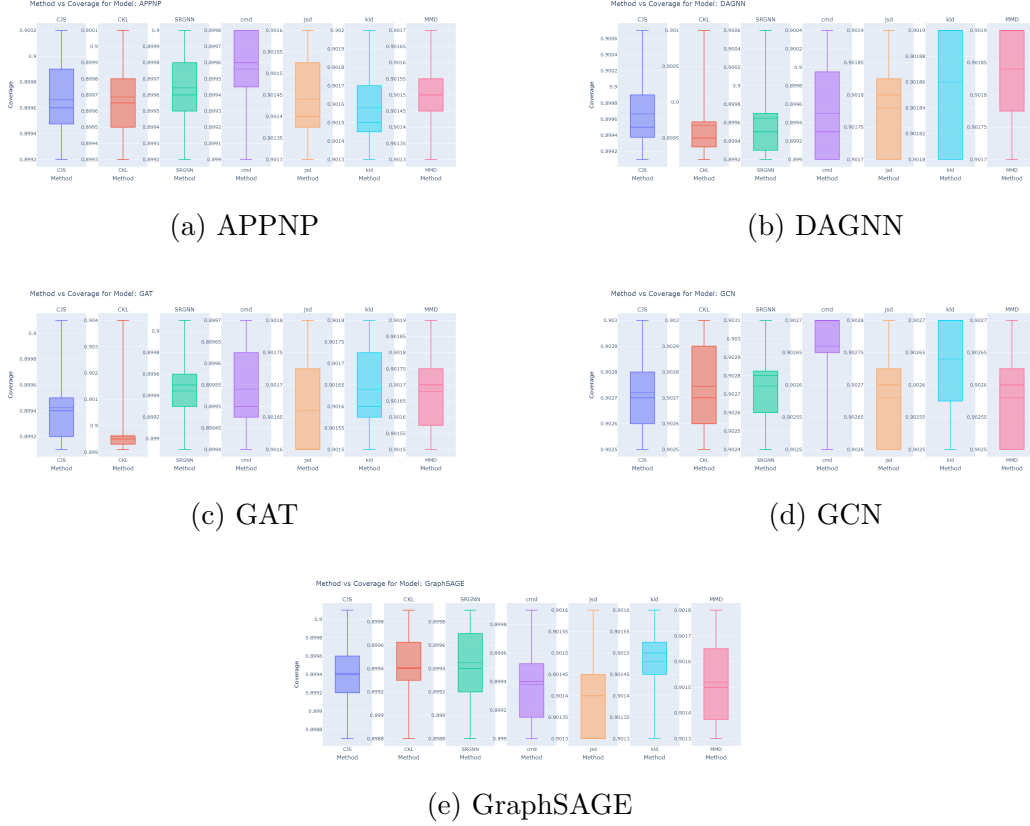


Figure 8.4: Method vs. Coverage for fixed models on the Cora dataset.

Above figure demonstrate the effects of various regularization methods on coverage metrics for fixed GNN architectures. CMD and MMD demonstrate broad and stable coverage distributions across architectures, highlighting their robustness in quantifying uncertainty under varying shift conditions. Among the models, GraphSAGE and APPNP achieve the most consistent coverage metrics, indicating their robustness in handling conditional shifts, while GCN shows greater variability, particularly with methods like JSD and KLD. Notably, CMD-inclusive methods such as CKL and SRGNN maintain wider interquartile ranges in coverage, reflecting their adaptability to diverse conditions while ensuring reliability. In contrast, single-metric approaches like JSD, though yielding tighter distributions, lack flexibility and robustness under conditional shifts. These findings emphasise

the importance of CMD-inclusive methods in achieving reliable model calibration and align with the report’s focus on robust uncertainty quantification.

1.5 Method vs. Prediction Set Size (Fix Model, Vary Methods)

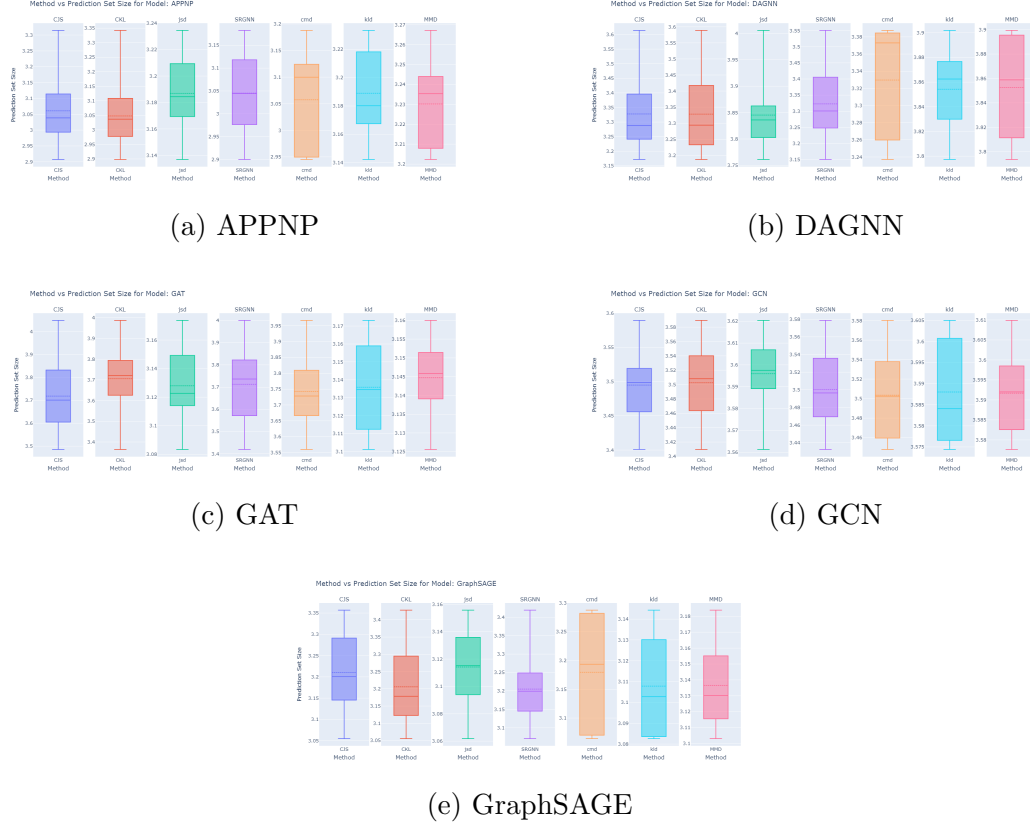


Figure 8.5: Method vs. Prediction Set Size for fixed models on the Cora dataset.

This plot shows the influence of different regularization methods on prediction set sizes when GNN architectures are fixed. CMD-based methods such as CKL (CMD + KLD) and SRGNN (CMD + MMD) achieve optimized and stable prediction set sizes, demonstrating their efficacy in regularizing across diverse architectures. APPNP and DAGNN consistently outperform GCN and GAT by achieving smaller and more informative prediction sets, reflecting their superior designs for local-global information aggregation. In contrast, simpler metrics like JSD and KLD exhibit significant variability in prediction set sizes, highlighting their sensitivity to architectural constraints and limited ability to

address conditional shifts. GraphSAGE demonstrates balanced performance across all methods, while GCN’s effectiveness appears more dependent on the chosen regularization method. These results illustrate the critical role of CMD-inclusive frameworks in optimizing prediction set sizes and further validate the utility of combined regularization strategies in addressing conditional shifts.

1.6 Prediction Set Size vs. Methods

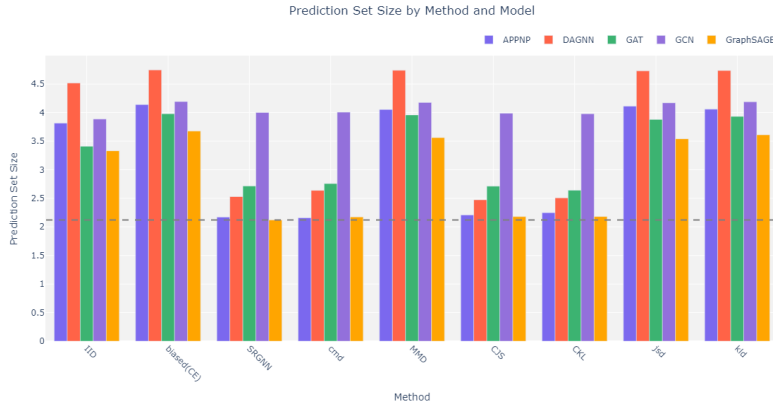


Figure 8.6: Comparative analysis of prediction set sizes across different regularization methods for various graph neural network architectures on the Cora dataset.

Figure 8.6 presents a comprehensive comparison of prediction set sizes across different regularization methods and GNN architectures for the Cora dataset. The bar plot reveals significant variations in how different regularization approaches influence the informativeness of prediction sets. CMD methods, particularly SRGNN and CKL, consistently achieve more compact prediction sets across multiple architectures, demonstrating their effectiveness in producing more precise uncertainty estimates. Among the evaluated models, APPNP achieves notably smaller prediction sets when combined with CMD and SRGNN regularization, suggesting superior uncertainty quantification capabilities. KLD-based approaches

show considerable variability across architectures, yielding compact prediction sets for DAGNN but larger ones for GCN, highlighting the architecture-specific nature of certain regularization effects. The JSD method produces relatively consistent prediction set sizes across different architectures, though generally larger than those achieved by CMD-inclusive approaches. GraphSAGE exhibits balanced performance across various regularization techniques, indicating its inherent robustness to different forms of regularization. Interestingly, CE (Cross-Entropy) regularization results in consistently larger prediction sets across most architectures, underscoring its limitations as a standalone approach for addressing conditional shifts. These findings substantiate the thesis proposition that combined regularization strategies, especially those incorporating CMD, can significantly enhance the informativeness of prediction sets by effectively mitigating distributional shifts in graph-structured data, thereby improving the practical utility of uncertainty estimates in real-world applications.

2 Citeseer

2.1 Accuracy vs. Method

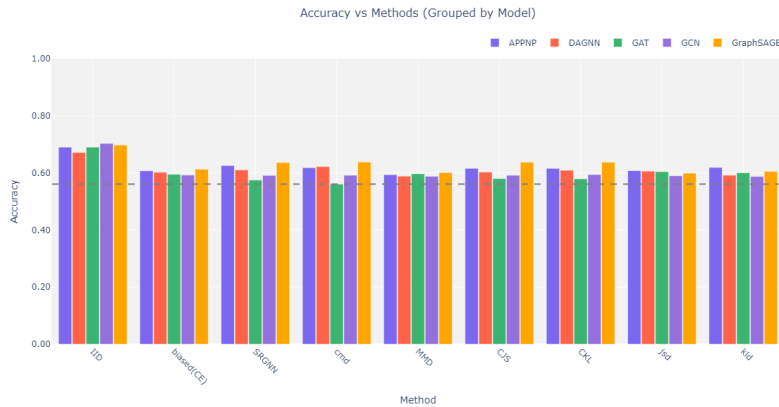


Figure 8.7: Comparative analysis of accuracy across different regularization methods for various graph neural network architectures.

Figure 8.7 presents an analysis of accuracy across various regularization methods for five distinct graph neural network architectures (APPNP, DAGNN, GAT, GCN, and GraphSAGE) on the Citeseer dataset. The visualization reveals that among regularization approaches, SRGNN (CMD+MMD) yields the highest accuracies across most architectures, highlighting the synergistic effect of combining complementary distance metrics. Notably, CKL (CMD+KLD) and CJS (CMD+JSD) exhibit comparable performance to SRGNN, reinforcing the effectiveness of CMD-inclusive regularization frameworks in addressing conditional shifts. The performance differential between architectures narrows under CMD-based methods, suggesting these approaches help standardize model behavior regardless of architectural differences. APPNP consistently achieves superior accuracy when paired with CMD-inclusive methods, particularly SRGNN, demonstrating its enhanced ability to leverage combined metrics for effective feature alignment. DAGNN follows APPNP in performance, further confirming that architectures with sophisticated propagation mechanisms better utilize regularization techniques to mitigate conditional shifts. Single-metric approaches such as MMD and CMD show intermediate performance, while JSD and KLD tend to yield marginally lower accuracies, indicating their limited capacity to address complex distributional shifts independently. These findings align with our central thesis that integrating multiple distance metrics within a unified regularization framework significantly enhances model robustness under conditional shifts in graph-structured data.

2.2 Accuracy vs. Prediction Set Size (Fix Method, Vary Model)

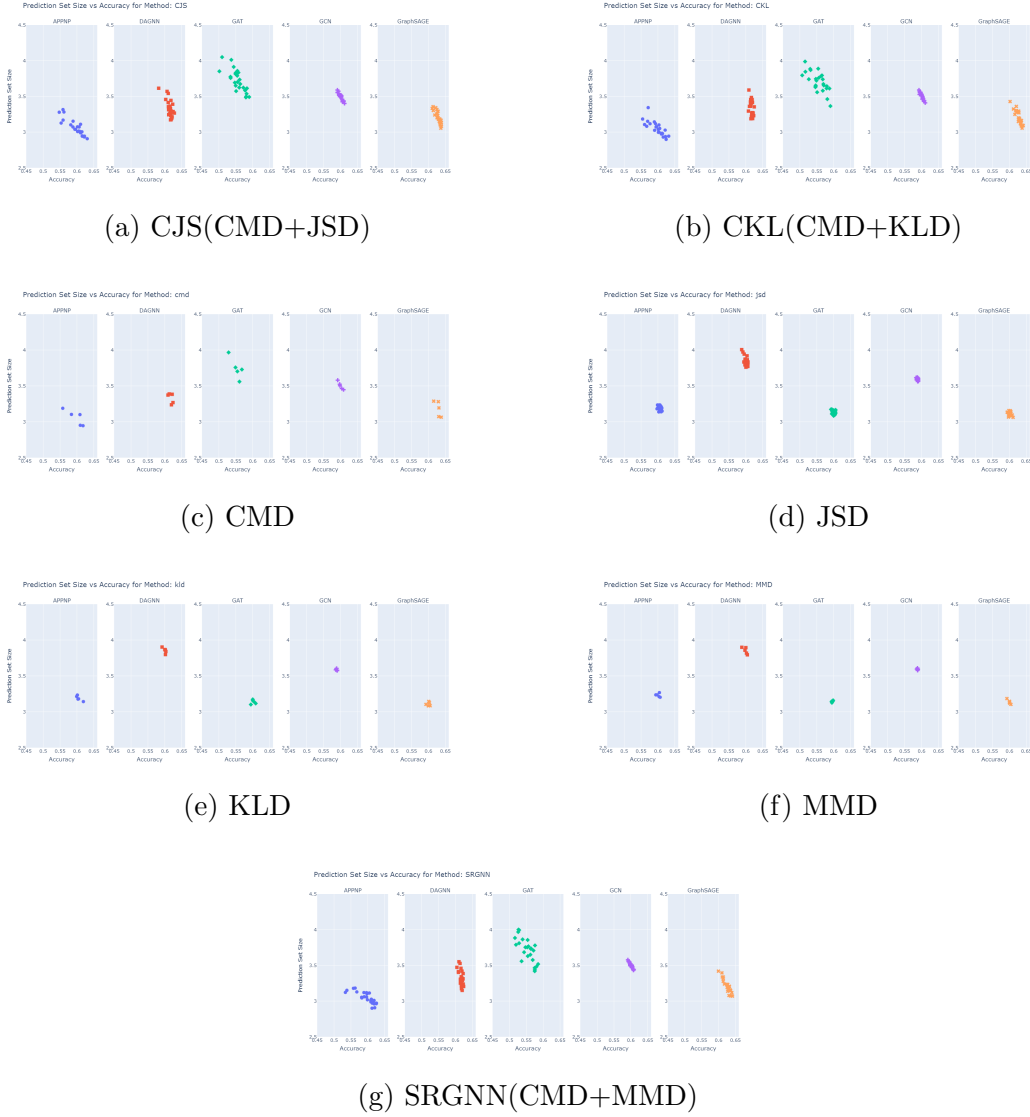


Figure 8.8: Accuracy vs. Prediction Set Size for fixed method and varying models.

Figure 8.8 explores how prediction set size relates to accuracy when keeping regularization methods constant while changing architectures. Combined regularization approaches—particularly CJS and SRGNN—create clear, well-defined clusters with minimal overlap, showing their ability to produce consistent results across

different architectures. In these plots, APPNP and DAGNN form clusters in the upper regions with moderate prediction set sizes, showing they can maintain high accuracy while keeping prediction sets manageable. In contrast, single-metric methods produce scattered, less organised patterns that indicate unstable relationships between accuracy and prediction set size. CMD alone performs better than JSD or KLD individually, but still falls short of combined approaches. KLD shows particular sensitivity to architectural differences, with widely dispersed data points across the accuracy spectrum. MMD performs somewhat better but lacks the consistency of combined methods. The visualization clearly shows how methods that integrate CMD with other metrics help standardise performance across different architecture types, reducing the impact of architectural limitations. This pattern strongly backs our proposition that combining complementary distance metrics creates a more effective regularization strategy that works across diverse network designs.

2.3 Accuracy vs. Prediction Set Size (Fix Model and Vary Methods)

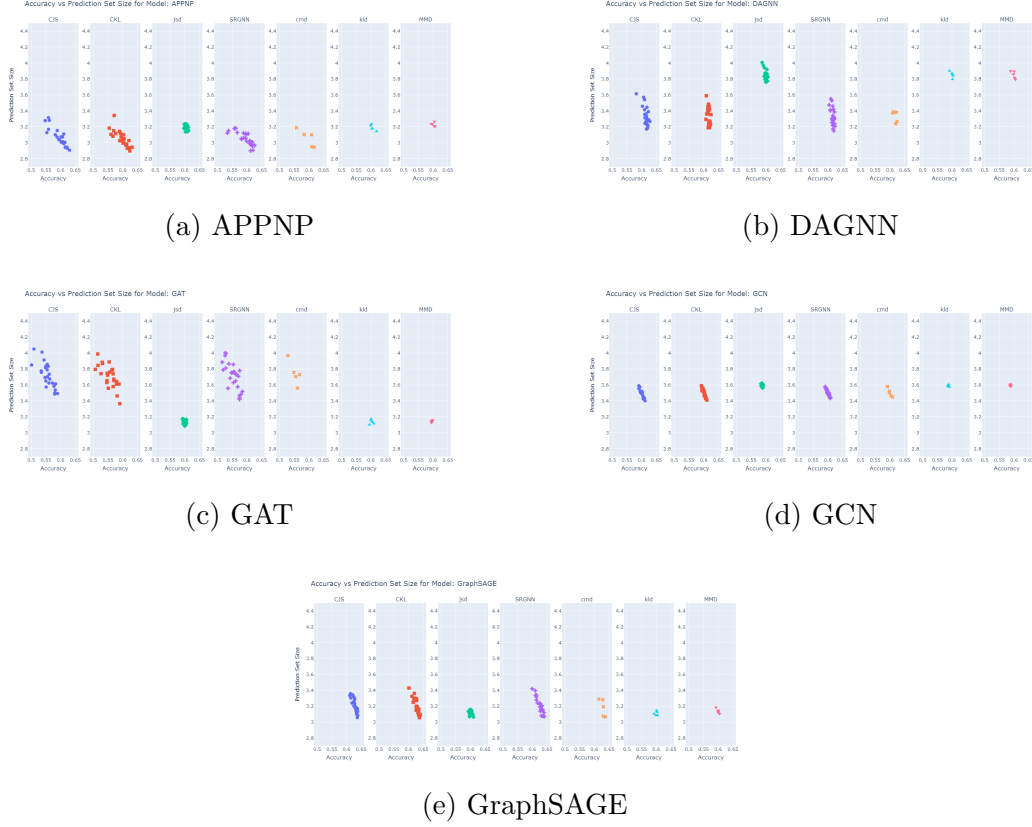
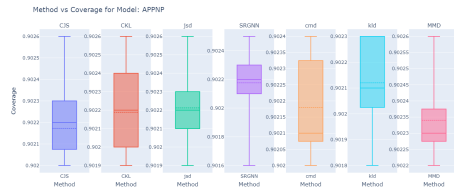


Figure 8.9: Accuracy vs. Prediction Set Size for fixed models with varying regularization methods.

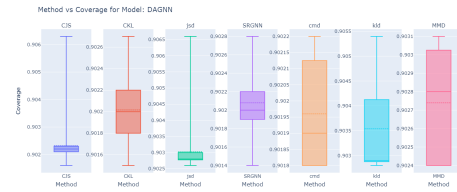
Figure 8.9 reveals how different regularization approaches affect each GNN architecture’s performance on Citeseer data. APPNP shows a striking pattern where CMD-based methods cluster distinctly in the upper portion of the plot, achieving both higher accuracy and smaller prediction sets compared to other approaches. DAGNN displays even clearer separation between method types, with SRGNN and CKL forming tight clusters that outperform single-metric methods. GCN and GAT show much more method-dependent behavior, with widely varying performance based on the regularization used, suggesting these architectures rely more heavily on appropriate regularization. GraphSAGE

falls between these extremes, showing moderate method sensitivity. A consistent finding across all architectures is that methods combining CMD with other metrics (especially KLD and MMD) help models maintain accuracy while generating more informative prediction sets. Single-metric approaches create less favorable accuracy-to-prediction set size ratios, particularly for the more complex architectures. These results reinforce our finding that integrated regularization frameworks address conditional shifts more effectively by balancing prediction accuracy with uncertainty estimation, though the specific benefits vary based on architectural design.

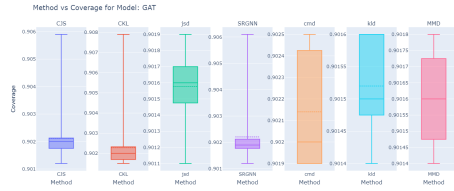
2.4 Method vs. Coverage (Fix Model)



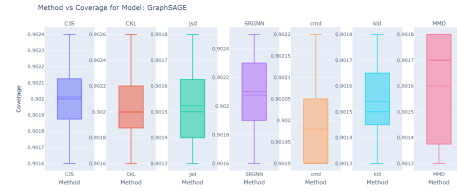
(a) APPNP



(b) DAGNN



(c) GAT



(d) GCN



(e) GraphSAGE

Figure 8.10: Method vs. Coverage for fixed models.

Figure 8.10 uses box plots to compare how different regularization methods affect coverage metrics across GNN architectures. For APPNP, methods that include CMD produce higher median coverage values with tighter distributions, indicating more reliable uncertainty estimates. DAGNN shows an interesting pattern where combined methods create wider coverage ranges with higher medians, suggesting these approaches allow the model to adapt better to varying data conditions. GAT displays the most dramatic differences between regularization types, with single-metric approaches creating erratic coverage distributions, showing their limitation in providing consistent uncertainty estimates. GCN’s results emphasise how heavily this architecture depends on proper regularization, with substantial improvements when using CMD-based methods. GraphSAGE demonstrates unusual stability across methods but still benefits from combined approaches. Looking at all architectures together reveals that CMD-inclusive methods consistently produce more balanced coverage distributions with appropriate medians and reasonable interquartile ranges. SRGNN frequently delivers the best coverage characteristics, followed closely by CKL. These plots support our argument that integrating multiple distance metrics leads to better-calibrated models that can provide more trustworthy uncertainty estimates when facing conditional shifts in graph data.

2.5 Method vs. Prediction Set Size (Fix Model, Vary Methods)

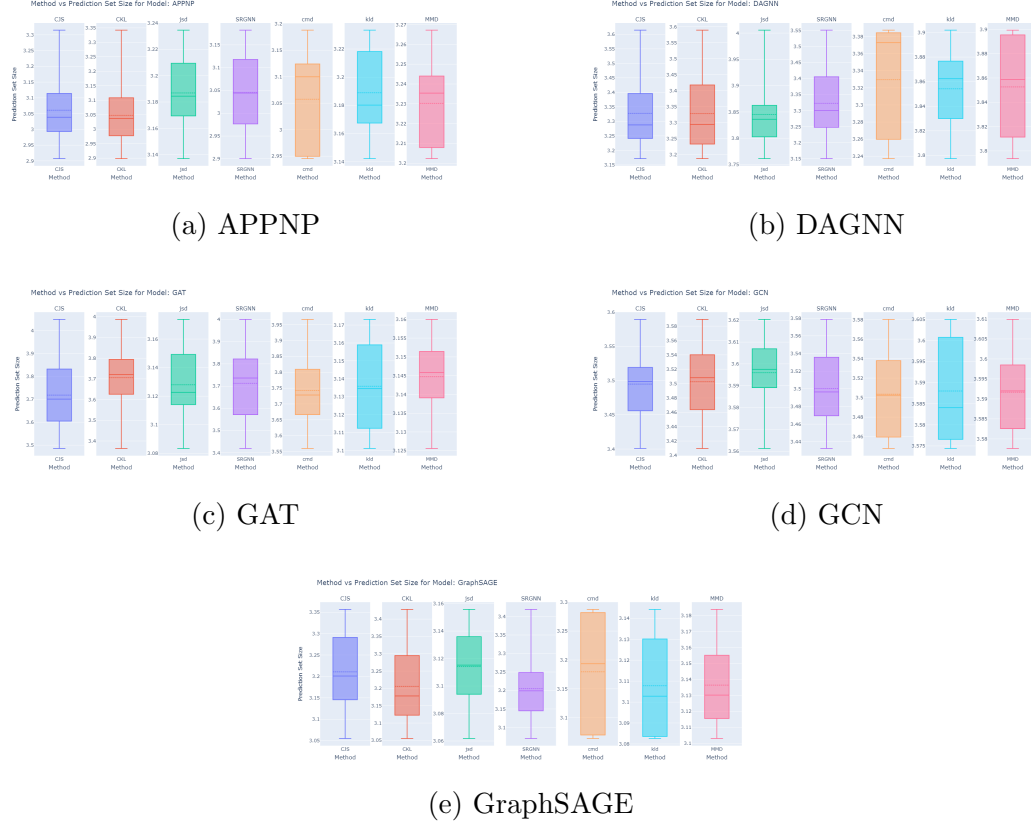


Figure 8.11: Method vs. Prediction Set Size for fixed models.

Figure 8.11 examines how prediction set sizes change with different regularization methods across fixed architectures. APPNP maintains relatively similar prediction set sizes regardless of method, though combined approaches produce slightly more compact distributions. DAGNN shows greater method sensitivity, with CKL and SRGNN creating notably smaller prediction sets compared to JSD or KLD alone. For GAT, the differences between methods become even more pronounced—combined approaches reduce prediction set sizes substantially, making uncertainty estimates more informative and useful. GCN exhibits extreme sensitivity to regularization choice, with combined methods drastically

improving prediction set compactness compared to single-metric alternatives. GraphSAGE shows moderate variation across methods but maintains the pattern of CMD-inclusive approaches producing smaller, more consistent prediction sets. Across all architectures, methods that pair CMD with complementary metrics consistently generate more informative prediction sets with reduced variability. This effect becomes more pronounced as architectural complexity increases, suggesting that sophisticated regularization becomes increasingly important for complex models. These findings validate our approach of developing integrated regularization frameworks that combine the strengths of different distance metrics to optimize the precision of uncertainty estimates.

2.6 Prediction Set Size vs. Methods (Bar Plot)

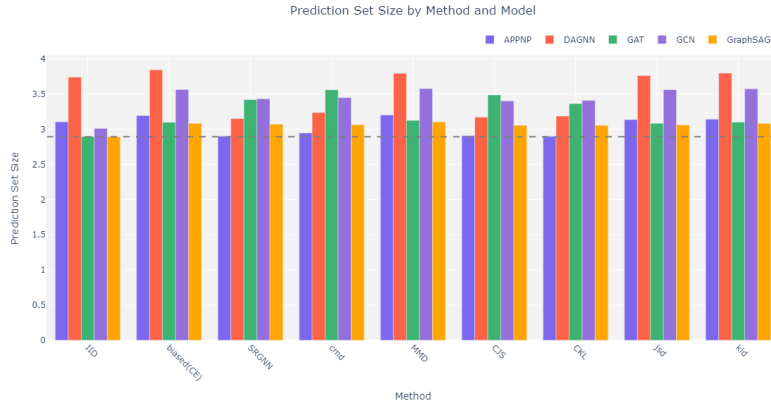


Figure 8.12: Comparative analysis of prediction set sizes across different regularization methods for various graph neural network architectures.

Figure 8.12 provides a direct comparison of prediction set sizes across both regularization methods and architectures for the Citeseer dataset. The visualisation shows that DAGNN consistently produces the smallest prediction sets across most regularization approaches, highlighting its superior uncertainty quantification abilities. CKL and SRGNN stand out among regularization methods by generating more compact prediction sets across all architectures, reinforcing

the value of combined metric approaches. GCN shows dramatic variation in prediction set sizes depending on the regularization used, making it the most method-dependent architecture. Single-metric approaches—particularly JSD and KLD—generally produce larger prediction sets that contain less specific information about model uncertainty. APPNP exhibits remarkable consistency across methods, though it still benefits from combined approaches. GAT shows moderate improvement with CMD-inclusive methods but remains less efficient than APPNP or DAGNN at generating compact prediction sets. The inverse relationship between architecture complexity and prediction set consistency suggests that simpler models benefit more substantially from sophisticated regularization. These results provide compelling evidence that our approach of combining complementary distance metrics within a unified framework leads to more precise uncertainty quantification across diverse network designs, with particular benefits for architectures that effectively balance local and global graph information.

Chapter 9

Conclusion

This work extends robust prediction frameworks for GNNs to address distributional shifts in the inductive setting, leveraging dual-metric loss functions that balance accuracy and alignment. Visualisations revealed that combining CMD with metrics such as KLD, JSD, or MMD significantly outperforms their standalone use, capturing complementary aspects of data divergence, ensuring better distributional alignment by obtaining higher accuracy, lower prediction set size and enhanced prediction reliability. This approach improves both model robustness and subjective evaluation criteria, such as uncertainty quantification and efficiency, in semi-supervised node classification tasks under conditional shifts.

Future research will explore extending the framework to complex, multi-modal datasets and dynamically adapting the loss components to evolving data distributions. These advancements aim to broaden the framework’s impact across diverse application domains and more challenging distributional scenarios.

Appendix A

Appendix

1 Hyperparameters

The main hyperparameters for our proposed method are λ_1 and λ_2 . Through empirical analysis we found the optimal parameter are as follows: For Cora, SRGNN achieves its best balance of high accuracy and minimal prediction set size at $\lambda_1 = 0.5$, $\lambda_2 = 1.0$, resulting in an accuracy of 0.7522, 90% coverage of 0.8995, and a mean prediction set size of 2.1720. CKL attains optimal results at $\lambda_1 = 0.7$, $\lambda_2 = 0.3$ with an accuracy of 0.7447, 90% coverage of 0.8997, and a mean set size of 2.2506, while CJS is best at $\lambda_1 = 0.3$, $\lambda_2 = 0.7$, yielding 0.7456 accuracy, 0.8996 coverage, and the smallest mean prediction set size of 2.2091 among the three. For Citeseer, SRGNN performs optimally at $\lambda_1 = 0.1$, $\lambda_2 = 1.0$, with an accuracy of 0.6412, 90% coverage of 0.9001, and a mean set size of 2.3831. Both CKL and CJS reach their best performance at $\lambda_1 = 0.1$, $\lambda_2 = 0.3$, with CKL achieving 0.6405 accuracy, 0.8999 coverage, and a mean set size of 2.3812, and CJS achieving 0.6384 accuracy, 0.8997 coverage, and the smallest mean set size of 2.3769. These results demonstrate that careful tuning of λ_1 and λ_2 enables each model to achieve competitive accuracy and coverage

while minimizing the prediction set size, with CJS consistently providing the most compact prediction sets across both datasets.

Bibliography

- [1] L. Wu, P. Cui, J. Pei, L. Zhao, and X. Guo, “Graph neural networks: foundation, frontiers and applications,” in *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, 2022, pp. 4840–4841.
- [2] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” *Advances in neural information processing systems*, vol. 30, 2017.
- [3] I. Chami, S. Abu-El-Haija, B. Perozzi, C. Ré, and K. Murphy, “Machine learning on graphs: A model and comprehensive taxonomy,” *Journal of Machine Learning Research*, vol. 23, no. 89, pp. 1–64, 2022.
- [4] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lió, and Y. Bengio, “Graph attention networks,” in *ICLR*, 2018.
- [5] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-i. Kawarabayashi, and S. Jegelka, “Representation learning on graphs with jumping knowledge networks,” in *International conference on machine learning*. PMLR, 2018, pp. 5453–5462.
- [6] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” *arXiv preprint arXiv:1810.00826*, 2018.

- [7] P. W. Koh, S. Sagawa, H. Marklund, S. M. Xie, M. Zhang, A. Balsubramani, W. Hu, M. Yasunaga, R. L. Phillips, I. Gao *et al.*, “Wilds: A benchmark of in-the-wild distribution shifts,” in *International conference on machine learning*. PMLR, 2021, pp. 5637–5664.
- [8] J. Quinonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, *Dataset shift in machine learning*. The MIT Press, 2009.
- [9] A. J. Storkey, “When training and test sets are different: characterising learning transfer,” in *Dataset shift in machine learning*. The MIT Press, 2009, pp. 3–28.
- [10] R. J. Tibshirani, R. Foygel Barber, E. Candes, and A. Ramdas, “Conformal prediction under covariate shift,” *Advances in neural information processing systems*, vol. 32, 2019.
- [11] F. Wang, Y. Liu, K. Liu, Y. Wang, S. Medya, and P. S. Yu, “Uncertainty in graph neural networks: A survey,” *arXiv preprint arXiv:2403.07185*, 2024.
- [12] M. Abdar, F. Pourpanah, S. Hussain, D. Rezazadegan, L. Liu, M. Ghavamzadeh, P. Fieguth, X. Cao, A. Khosravi, U. R. Acharya *et al.*, “A review of uncertainty quantification in deep learning: Techniques, applications and challenges,” *Information Fusion*, vol. 76, pp. 243–297, 2021.
- [13] G. Shafer and V. Vovk, “A tutorial on conformal prediction,” *arXiv preprint arXiv:0706.3188*, 2007.
- [14] Z. Ghahramani, “Probabilistic machine learning and artificial intelligence,” *Nature*, vol. 521, no. 7553, pp. 452–459, 2015.

- [15] S. Akansha, “Conditional shift-robust conformal prediction for graph neural network,” *arXiv preprint arXiv:2405.11968*, 2024.
- [16] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic learning in a random world*. Springer Science & Business Media, 2005.
- [17] G. Shafer and V. Vovk, “A tutorial on conformal prediction,” *Journal of Machine Learning Research*, vol. 9, no. Mar, pp. 371–421, 2008.
- [18] A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” *arXiv preprint arXiv:2107.07511*, 2021.
- [19] H. Papadopoulos, A. Gammerman, and V. Vovk, “Conformal prediction with a confidence,” in *Fourth IEEE International Conference on Data Mining (ICDM’02)*. IEEE, 2002, pp. 371–378.
- [20] S. Kullback and R. A. Leibler, “On information and sufficiency,” *The Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 79–86, 1951.
- [21] W. Zellinger, T. Grubinger, E. Lughofer, T. Natschläger, and S. Saminger-Platz, “Central moment discrepancy (cmd) for domain-invariant representation learning,” *arXiv preprint arXiv:1702.08811*, 2017.
- [22] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, “The graph neural network model,” in *IEEE Transactions on Neural Networks*, vol. 20, no. 1. IEEE, 2009, pp. 61–80.
- [23] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, “A comprehensive survey on graph neural networks,” *IEEE transactions on neural networks and learning systems*, vol. 32, no. 1, pp. 4–24, 2020.

- [24] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pp. 1263–1272, 2017.
- [25] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: <https://arxiv.org/abs/1609.02907>
- [26] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://arxiv.org/abs/1710.10903>
- [27] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” *Advances in neural information processing systems*, vol. 30, 2017.
- [28] S. Bhagat, P. Shachaf, and L. Getoor, “Node classification in social networks,” *Social network data analytics*, pp. 115–148, 2011.
- [29] M. Zitnik, S. Wang, and J. Leskovec, “Modeling disease progression trajectories using graph neural networks,” *bioRxiv*, p. 343543, 2018.
- [30] Y. Dou, Z. Liu, L. Sun, Y. Peng, B. Sturt, G. Ding, and S. Y. Philip, “Enhancing graph neural network-based fraud detectors against camouflaged fraudsters,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 01, 2020, pp. 588–595.

- [31] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, “Collective classification in network data,” *AI magazine*, vol. 29, no. 3, pp. 93–93, 2008.
- [32] P. W. Koh, S. Sagawa, H. Marklund, A. S. Xie, A. Raghunathan, S. Santurkar, T. Ma, T. Icard, B. K. Li, A. Valid *et al.*, “Wilds: A benchmark of in-the-wild distribution shifts,” *Proceedings of the 38th International Conference on Machine Learning (ICML)*, vol. 139, pp. 5637–5647, 2021.
- [33] M. Zhu, D. He, C. Liu, and Q. Gan, “Shifting attention to graph structure in graph neural networks,” *arXiv preprint arXiv:2106.00503*, 2021.
- [34] Y. Ma, Y. Liu, J. Chen, S. Xue, and N. Shah, “Ood-gnn: Out-of-distribution generalization of graph neural networks,” in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (CIKM)*, 2021, pp. 1253–1262.
- [35] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” *Proceedings of the 34th International Conference on Machine Learning (ICML)*, vol. 70, pp. 1321–1330, 2017.
- [36] S. Akansha and Others, “Title of akansha et al. 2024 paper on cp for gnns,” Conference or Journal or arXiv preprint, 2024, replace with actual publication details.
- [37] J. Lin, “Divergence measures based on the shannon entropy,” *IEEE Transactions on Information theory*, vol. 37, no. 1, pp. 145–151, 1991.
- [38] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *Journal of Machine Learning Research*, vol. 13, no. Mar, pp. 723–773, 2012.

- [39] W. Zellinger, T. Grubinger, E. Lughofer, T. Natschläger, and S. Saminger-Platz, “Central moment discrepancy (cmd) for domain-invariant representation learning,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [40] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016.
- [41] D. K. Hammond, P. Vandergheynst, and R. Gribonval, “Wavelets on graphs via spectral graph theory,” *Applied and Computational Harmonic Analysis*, vol. 30, no. 2, pp. 129–150, 2011.
- [42] M. Liu, H. Gao, and S. Ji, “Towards deeper graph neural networks,” in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 338–348.
- [43] S. Brin, “The pagerank citation ranking: bringing order to the web,” *Proceedings of ASIS, 1998*, vol. 98, pp. 161–172, 1998.
- [44] J. Gasteiger, A. Bojchevski, and S. Günnemann, “Predict then propagate: Graph neural networks meet personalized pagerank,” *arXiv preprint arXiv:1810.05997*, 2018.
- [45] V. Vapnik, *The nature of statistical learning theory*. Springer science & business media, 2013.
- [46] H. Shimodaira, “Improving predictive inference under covariate shift by weighting the log-likelihood function,” *Journal of statistical planning and inference*, vol. 90, no. 2, pp. 227–244, 2000.
- [47] D. Zügner and S. Günnemann, “Certifiable robustness and robust training for graph convolutional networks,” in *Proceedings of the 25th ACM*

- SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 246–256.
- [48] D. Zhu, Z. Zhang, P. Cui, and W. Zhu, “Robust graph convolutional networks against adversarial attacks,” in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 1399–1407.
 - [49] S. Yan, Y. Xiong, and D. Lin, “Spatial temporal graph convolutional networks for skeleton-based action recognition,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.
 - [50] H. Zhao, S. Zhang, G. Wu, J. M. Moura, J. P. Costeira, and G. J. Gordon, “Adversarial multiple source domain adaptation,” *Advances in neural information processing systems*, vol. 31, 2018.
 - [51] T. Zhu, F. He, L. Zhang, Z. Niu, M. Song, and D. Tao, “Topology-aware generalization of decentralized sgd,” in *International Conference on Machine Learning*. PMLR, 2022, pp. 27 479–27 503.
 - [52] W. Jin, Y. Ma, X. Liu, X. Tang, S. Wang, and J. Tang, “Graph structure learning for robust graph neural networks,” in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 66–74.
 - [53] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, “Beyond homophily in graph neural networks: Current limitations and effective designs,” *Advances in neural information processing systems*, vol. 33, pp. 7793–7804, 2020.
 - [54] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, and X. Sun, “Measuring and relieving the over-smoothing problem for graph neural networks from the

- topological view,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 3438–3445.
- [55] T. M. Cover, *Elements of information theory*. John Wiley & Sons, 1999.
- [56] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *The Journal of Machine Learning Research*, vol. 13, no. 1, pp. 723–773, 2012.
- [57] B. K. Sriperumbudur, A. Gretton, K. Fukumizu, B. Schölkopf, and G. R. Lanckriet, “Hilbert space embeddings and metrics on probability measures,” *The Journal of Machine Learning Research*, vol. 11, pp. 1517–1561, 2010.
- [58] E. Tzeng, J. Hoffman, N. Zhang, K. Saenko, and T. Darrell, “Deep domain confusion: Maximizing for domain invariance,” *arXiv preprint arXiv:1412.3474*, 2014.
- [59] K. Huang, Y. Jin, E. Candes, and J. Leskovec, “Uncertainty quantification over graph with conformalized graph neural networks,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 26 699–26 721, 2023.
- [60] R. Andersen, F. Chung, and K. Lang, “Local graph partitioning using pagerank vectors,” in *2006 47th annual IEEE symposium on foundations of computer science (FOCS’06)*. IEEE, 2006, pp. 475–486.
- [61] Q. Zhu, N. Ponomareva, J. Han, and B. Perozzi, “Shift-robust gnns: Overcoming the limitations of localized graph training data,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 27 965–27 977, 2021.